학부 강의

고급 트리 모형

결정 트리에서 XGBoost, LightGBM, CatBoost까지

결정 트리 · 랜덤 포레스트 · AdaBoost · 그래디언트 부스팅

수식 없이 그림으로 따라가는 강의: 모든 개념을 그림으로 설명한다.
모든 차트는 작은 시뮬레이션 데이터에 실제로 모형을 적합해 그렸다.

강의 전체를 꿰는 세 가지 생각

이 세 가지를 기억하자. 이후의 슬라이드는 모두 이 중 하나를 자세히 다룬다

1

트리는 예/아니오 질문의 목록이다

트리 하나는 읽기 쉽지만 불안정하다. 데이터가 조금만 바뀌어도 트리가 달라진다.

2

트리를 많이 평균하면 분산이 줄어든다

배깅과 랜덤 포레스트는 깊은 트리를 병렬로 키워 평균한다.

3

부스팅은 오차를 고치는 작은 트리를 더해 간다

AdaBoost, 그래디언트 부스팅, XGBoost, LightGBM, CatBoost는 트리를 순차적으로 만든다.

트리 하나 → 병렬로 여러 트리 → 순차적으로 여러 트리

강의는 다섯 단계로 진행한다

각 단계는 앞 단계를 바탕으로 한다

1 트리

기본 단위

단순한 예/아니오 질문. 불안정하다.

CART

2 포레스트

병렬

깊은 트리를 많이 평균한다.

랜덤 포레스트

3 부스팅

순차

작은 트리가 앞선 오차를 고친다.

AdaBoost, GBM

4 GBDT 도구

규제

벌점, 결측값, 범주형 변수.

XGBoost 등

5 해석

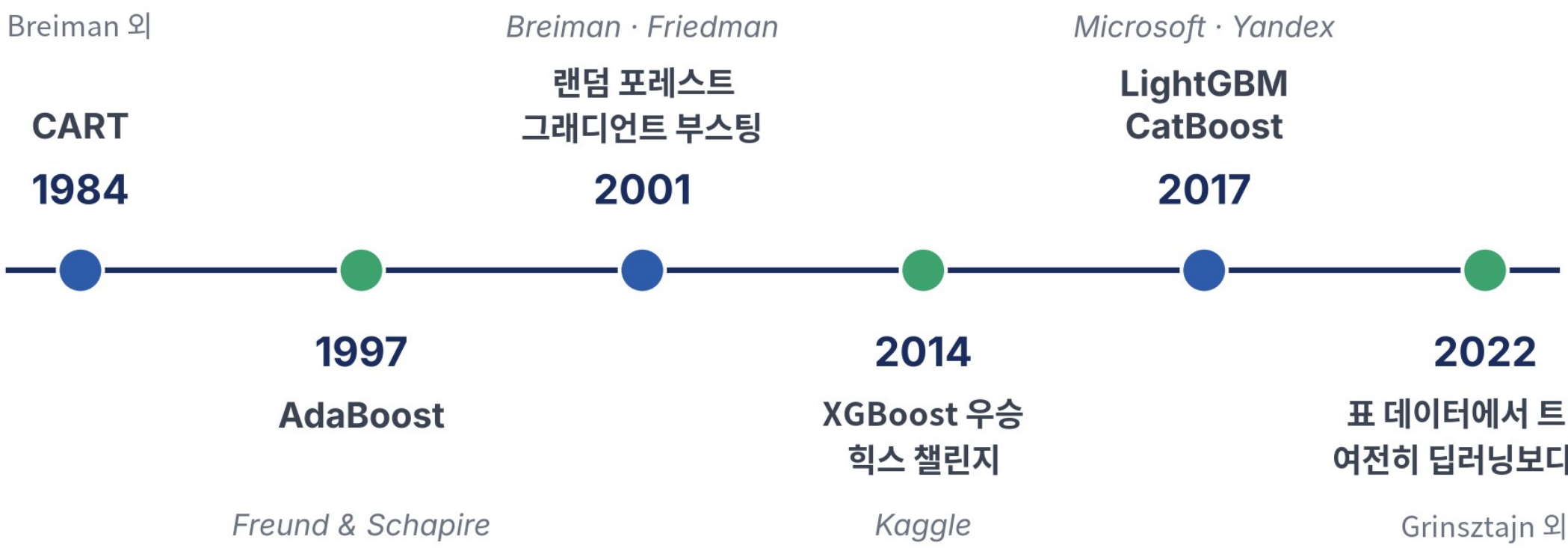
블랙박스 열기

어떤 입력이 왜 중요한가?

SHAP

CART에서 LightGBM까지, 트리 모형 40년

주요 논문과 사건. 2022년 연구에서도 표 형식 벤치마크 대부분에서 트리가 딥러닝보다 우수했다



01

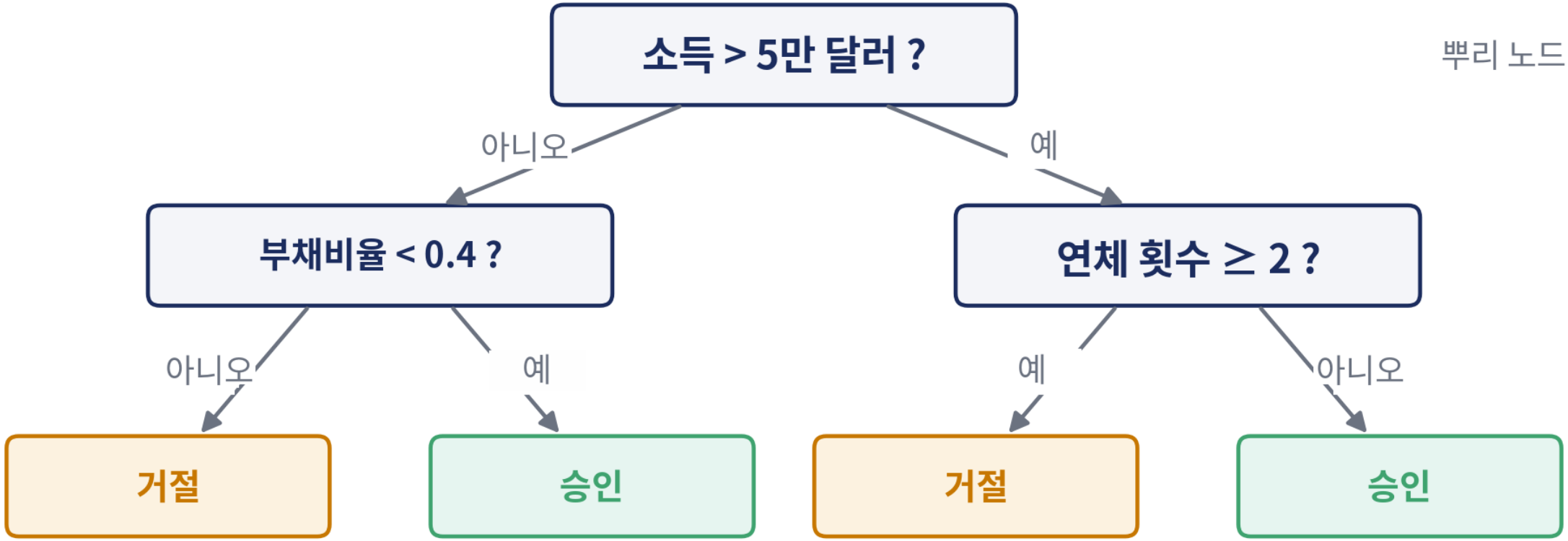
1부 · 결정 트리

결정 트리: 기본 단위

트리 하나가 질문하고, 분할을 고르고, 과적합하는 방식

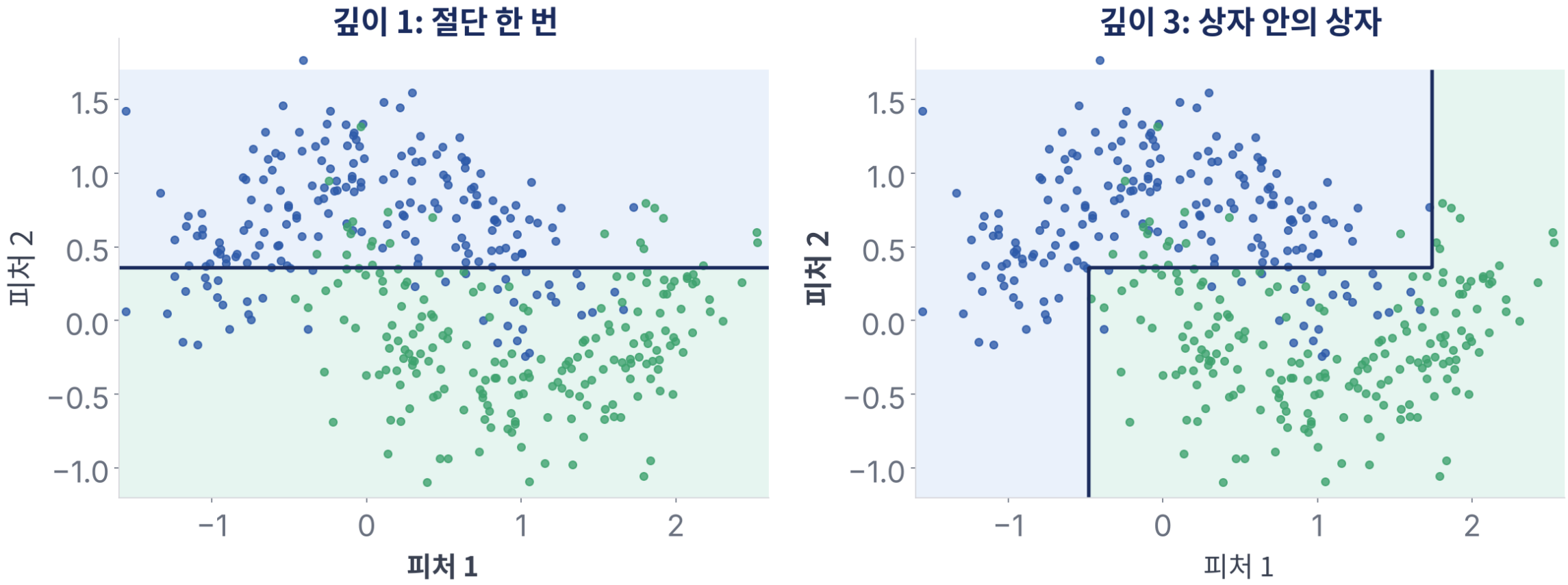
결정 트리는 예/아니오 질문을 차례로 던진다

대출 예시: 뿌리에서 출발해 답을 따라가고, 잎에서 예측을 읽는다



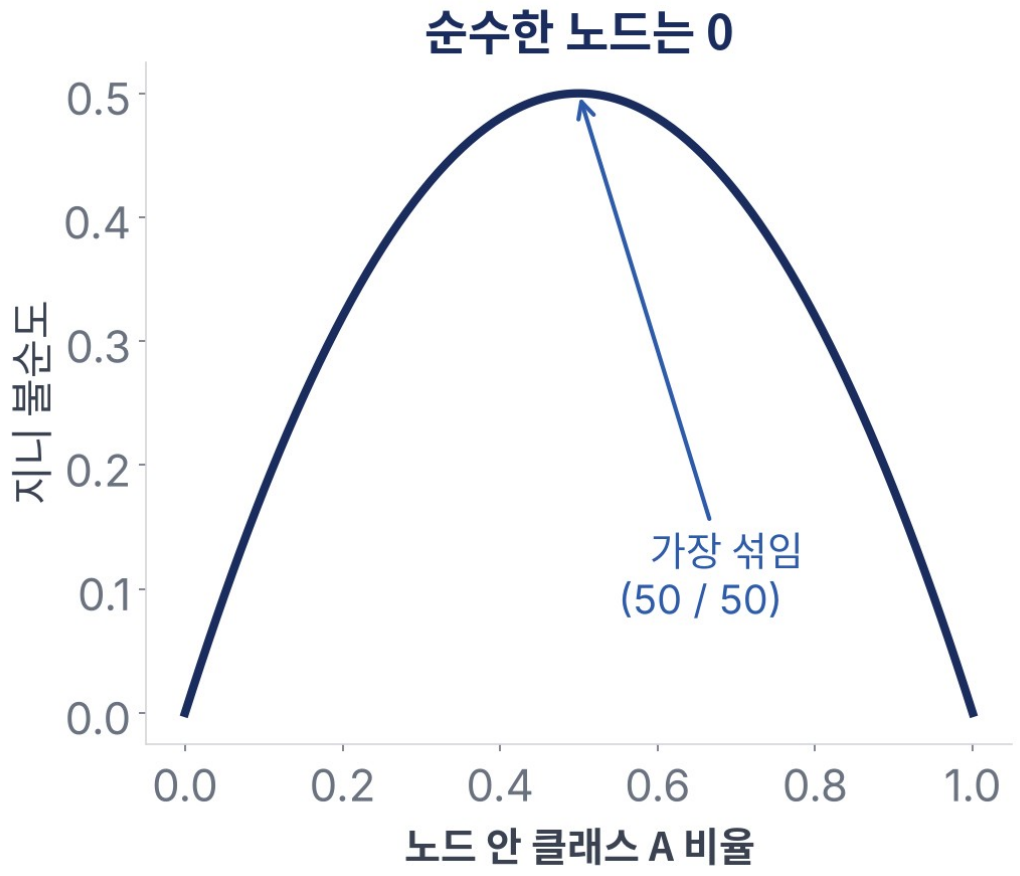
분할은 매번 피처 공간을 직선으로 자른다

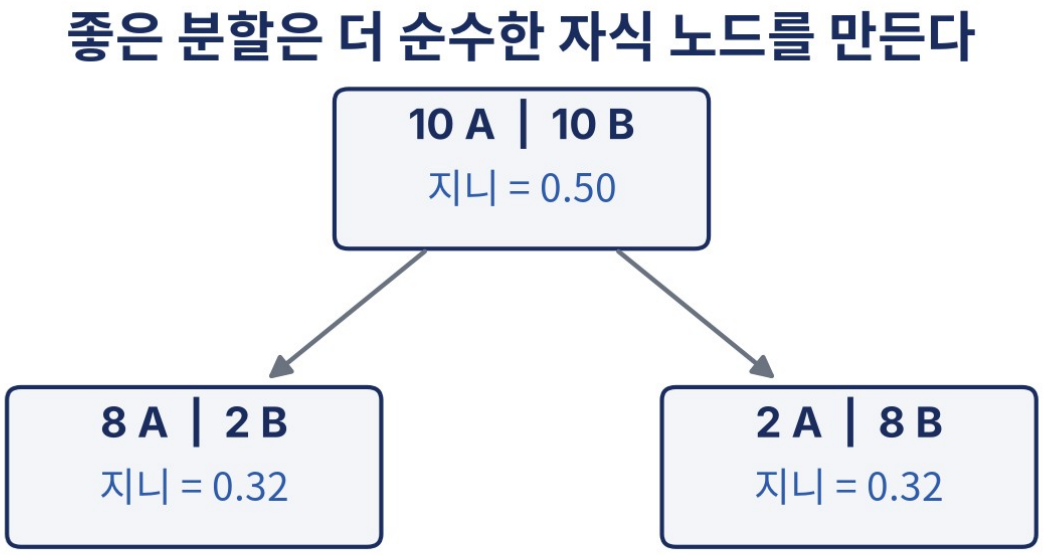
반달(two-moons) 데이터: 트리가 깊어질수록 축에 평행한 절단이 쌓여 직사각형이 된다



분할은 자식 노드가 더 순수해지도록 고른다

지니 불순도(Gini impurity)는 노드가 얼마나 섞였는지 잰다. 순수하면 0, 50:50일 때 가장 크다

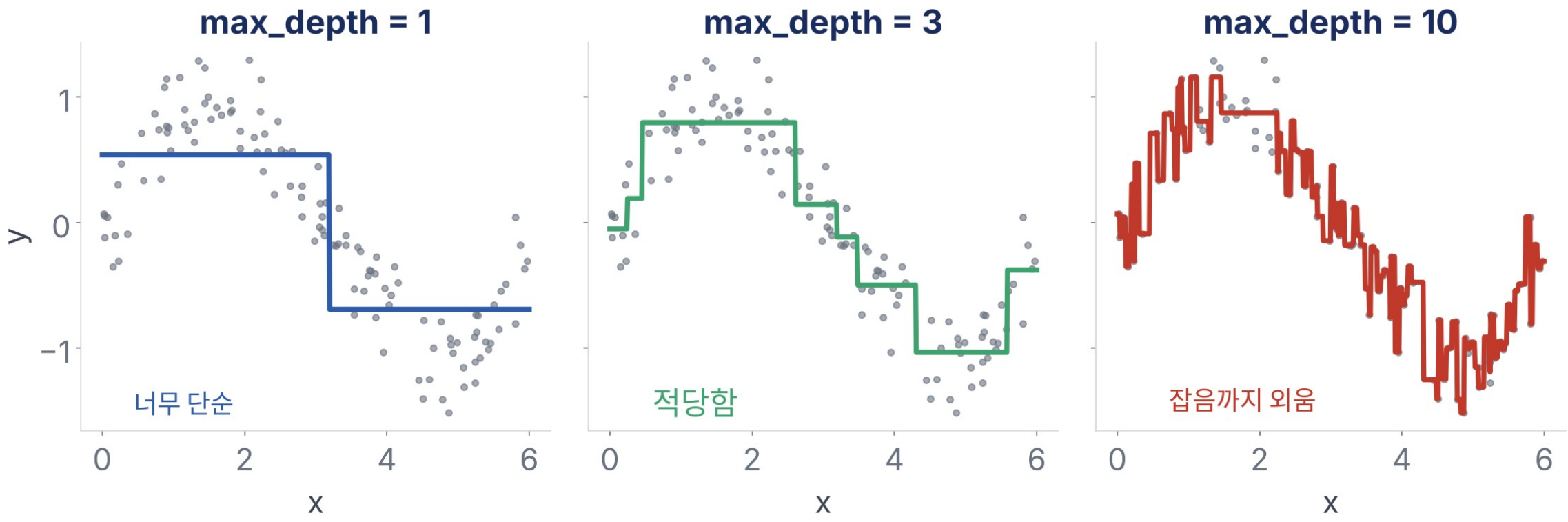




분할 후 가중 지니 = 0.32 → 불순도 감소 = 0.18

회귀 트리는 계단 함수로 예측한다

각 잎은 학습 데이터 y의 평균을 출력한다. 잡음이 섞인 사인 데이터 120개

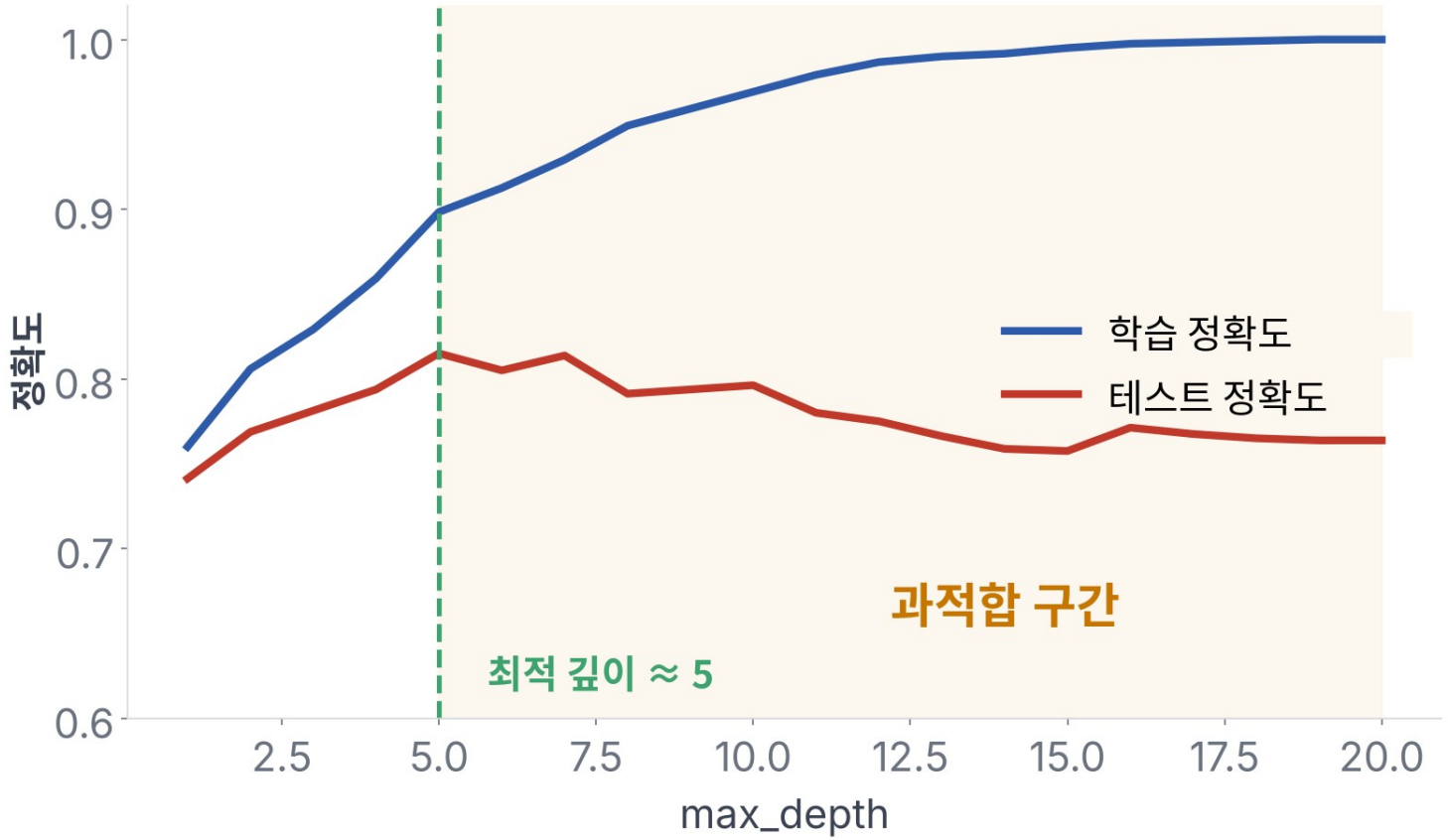


트리가 깊어지면 학습 데이터를 외운다

라벨 잡음 12%인 시뮬레이션 데이터, 정확도는 따로 떼어 둔 테스트 행에서 측정

읽는 법

- 학습 정확도는 100%까지 오른다.
- 테스트 정확도는 일찍 정점을 찍고 떨어진다.
- 이 차이가 과적합이다. 트리가 잡음을 학습한다.
- 처방: 깊이를 제한하거나, 가지치기하거나, 트리를 많이 평균한다.



트리가 얼마나 자랄지는 몇 가지 설정이 정한다

이름은 scikit-learn 기준이며 다른 라이브러리도 비슷한 설정을 쓴다

설정	제어 대상	키울 때 효과
max_depth	가장 긴 질문 경로	더 세밀, 과적합 증가
min_samples_leaf	허용하는 가장 작은 잎	더 매끄럽고 과적합 감소
max_features	분할마다 검토하는 피처 수	개별 분할이 더 정확
ccp_alpha	비용-복잡도 가지치기	트리 축소, 과적합 감소

경험칙: 트리 하나는 강하게 제한해야 하고, 포레스트는 가지치기하지 않은 깊은 트리를 써도 된다

에피소드

질문 세 개짜리 트리가 응급 분류에서 의사를 앞섰다

1990년대 시카고 쿡 카운티 병원: 리 골드먼(Lee Goldman)의 흉통 판단 규칙

심전도 결과 + 몇 가지 위험 요인 → 심장 전문 병동, 모니터링 병상, 귀가 중 하나로 배정

효과가 있었던 이유

- 질문이 적고, 매번 같은 방식으로 묻는다.
- 의사들은 너무 많은 정보를 일관성 없이 저울질했다.
- 규칙이 심근경색 위험을 더 안정적으로 분류했다.

교훈

- 짧은 트리는 투명하고 검증하기 쉽다.
- 그러나 트리 하나는 취약하다. 새 데이터에는 새 트리가 필요할 수 있다.
- 앙상블은 정확도를 얻는 대신 투명성을 일부 잃는다.

02

2부 · 배깅과 랜덤 포레스트

병렬로 키우는 여러 트리

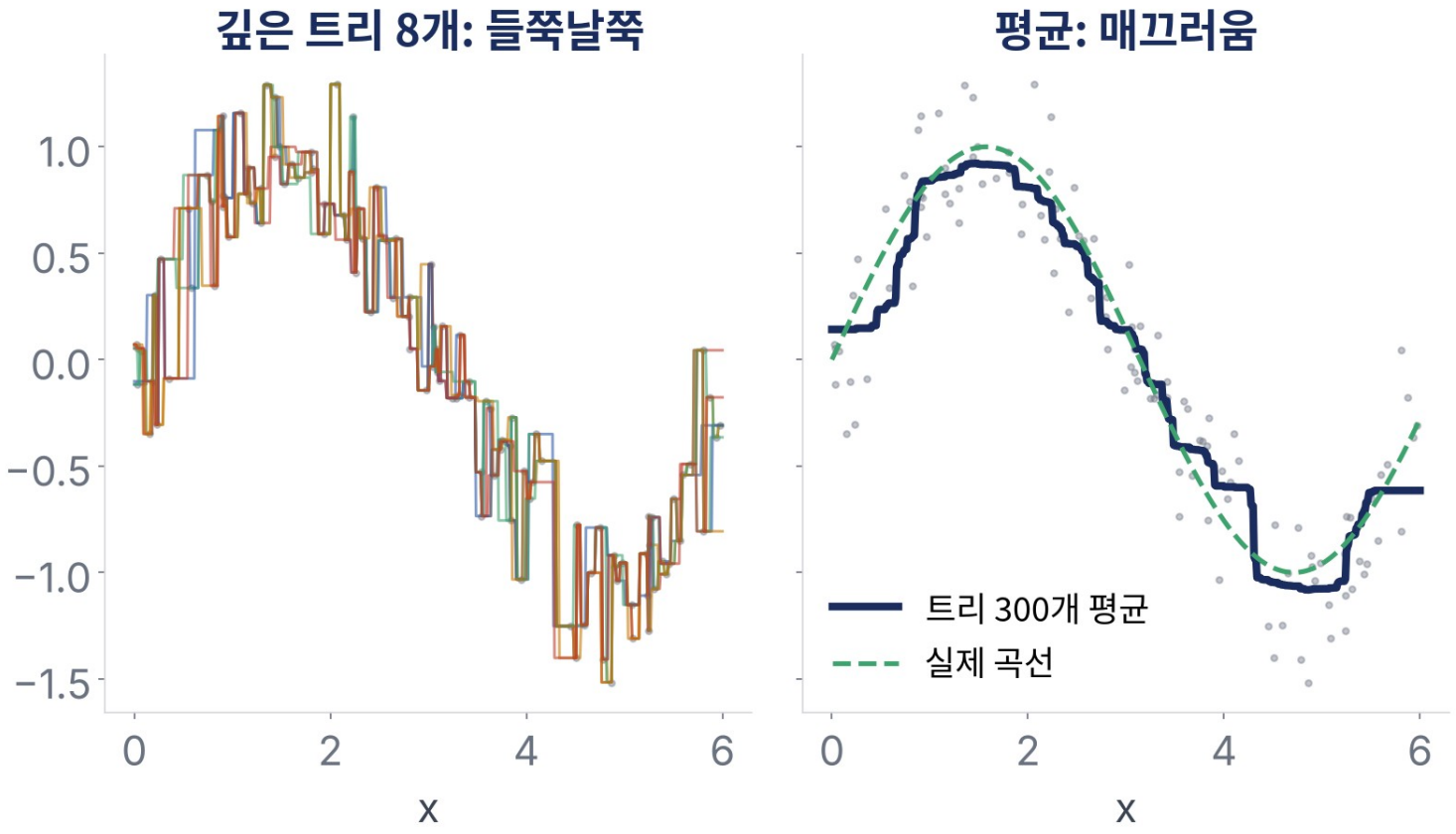
부트스트랩 표본, 무작위 피처, 평균의 힘

불안정한 트리를 많이 평균하면 잡음이 상쇄된다

왼쪽: 부트스트랩 표본 8개로 키운 깊은 트리. 오른쪽: 같은 데이터로 만든 트리 300개짜리 포레스트

핵심

- 깊은 트리는 편향이 낮지만 분산이 크다.
- 트리마다 오차의 방향이 다르다.
- 평균하면 신호는 남고 잡음은 상당 부분 상쇄된다.



배깅은 각 트리를 부트스트랩 표본으로 학습한다

부트스트랩 = n개 행을 복원추출. 뽑히지 않은 행은 그 트리의 OOB(out-of-bag) 행이 된다



n개 행을 복원추출 → 일부 행은 중복되고 약 37%는 빠진다(OOB)

랜덤 포레스트는 트리들이 일부러 서로 다르게 만든다

분할마다 무작위로 고른 소수의 피처만 보므로 트리가 서로 다르게 자란다

배깅: 모든 트리가 같은 방식으로 시작



트리들이 서로 닮았다
→ 같은 실수를 한다

평균의 효과가 작다

랜덤 포레스트: 트리마다 다르게 시작



분할마다 무작위로 고른 소수 피처만 본다
→ 실수가 서로 달라 상쇄된다

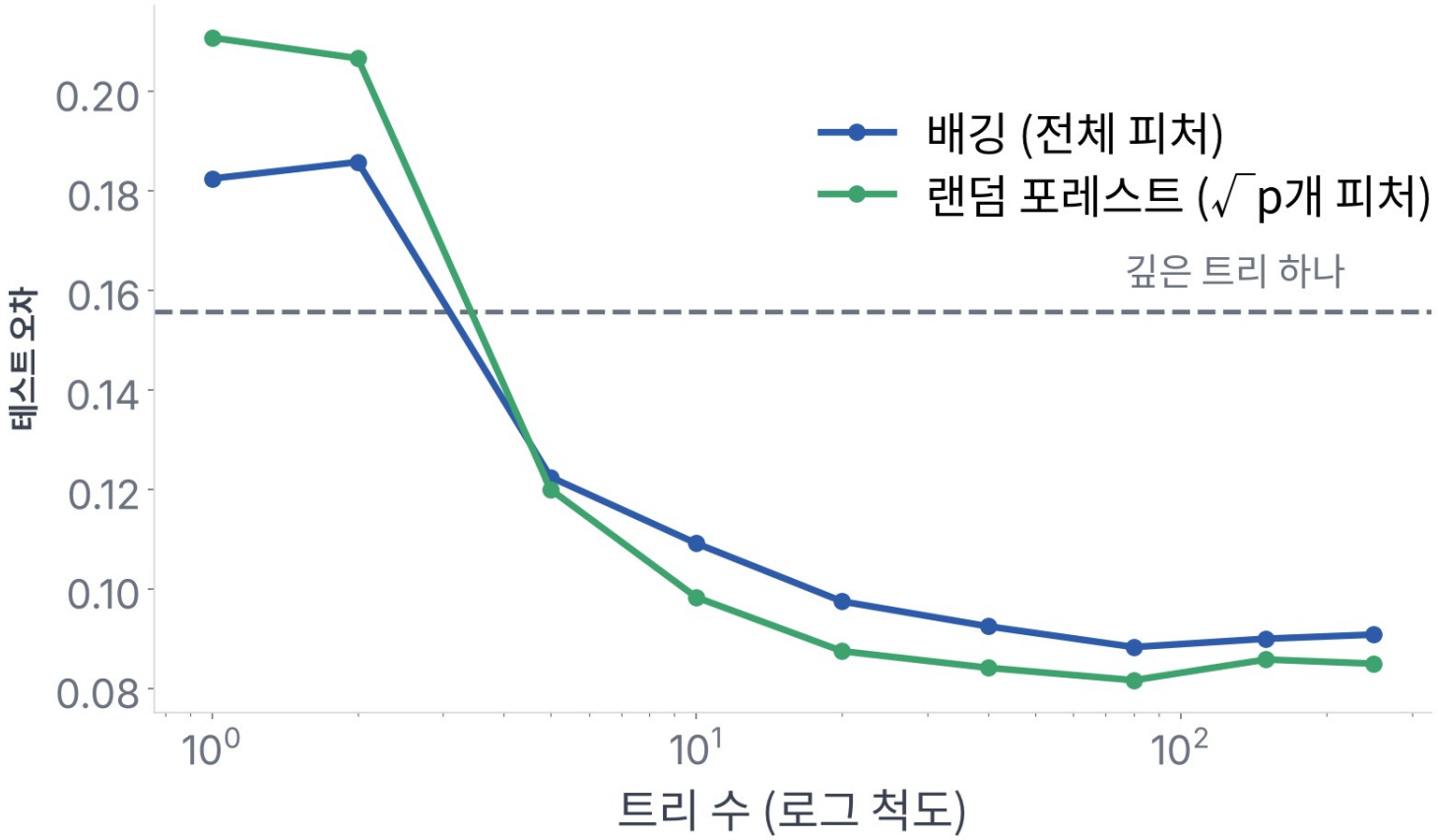
평균의 효과가 크다

무작위 피처는 배깅보다 조금 더 개선한다

시뮬레이션 데이터, 피처 30개. 트리 수에 따른 테스트 오차

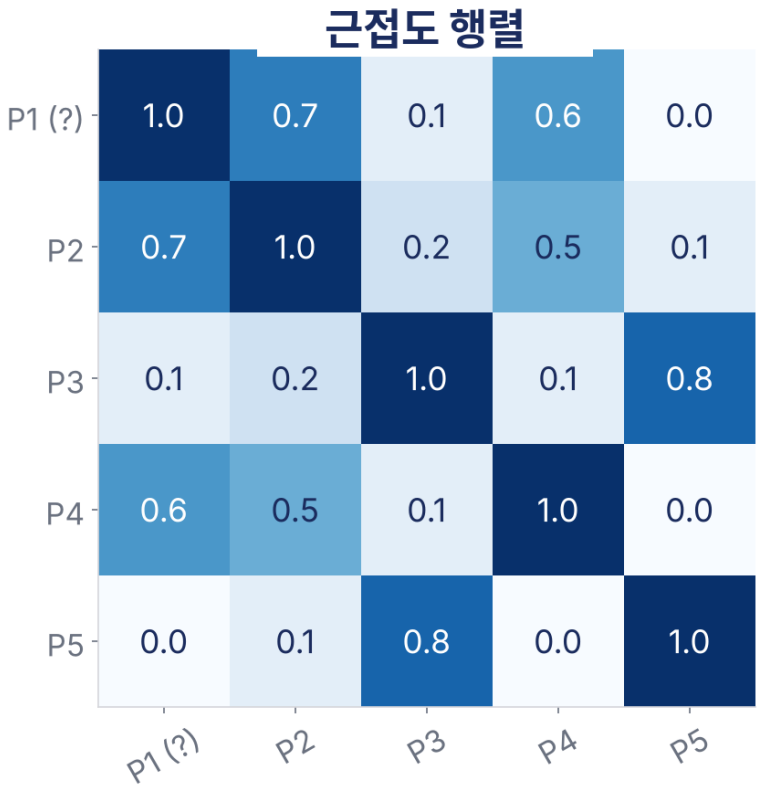
관찰

- 깊은 트리 하나: 오차 15.6%.
- 배깅 트리 250개: 9.1%.
- 트리 250개 랜덤 포레스트: 8.5%.
- 트리 약 100개 이후로는 개선이 거의 없다.



랜덤 포레스트는 비슷한 행에 물어 빈칸을 채운다

브라이먼(Breiman)의 생각: 자주 같은 잎에 들어가는 행은 비슷하므로 그 값을 빌려 온다



1 초기 추정 — 빈칸을 중앙값으로 채운다

2 포레스트 키우기 — 두 환자가 같은 잎에 들어간 횟수를 센다

3 가중 재채움 — 새 추정 = P2..P5 값의 근접도 가중평균

4 반복 — 추정값이 바뀌지 않을 때까지 다시 키우고 다시 채운다

1단계: 대략적인 추정값으로 시작한다

환자 4명의 예시 데이터. 첫 추정에는 라벨이 같은 환자만 쓴다

환자 4는 빈칸이 두 개

#	흉통	혈류 양호	동맥 막힘	몸무게	심장병
1	아니오	아니오	아니오	125	아니오
2	예	예	예	180	예
3	예	예	아니오	210	아니오
4	예	아니오	???	???	아니오

대략적인 첫 추정

#	흉통	혈류 양호	동맥 막힘	몸무게	심장병
1	아니오	아니오	아니오	125	**아니오**
2	예	예	예	180	예
3	예	예	아니오	210	**아니오**
4	예	아니오	**아니오**	**167.5**	아니오

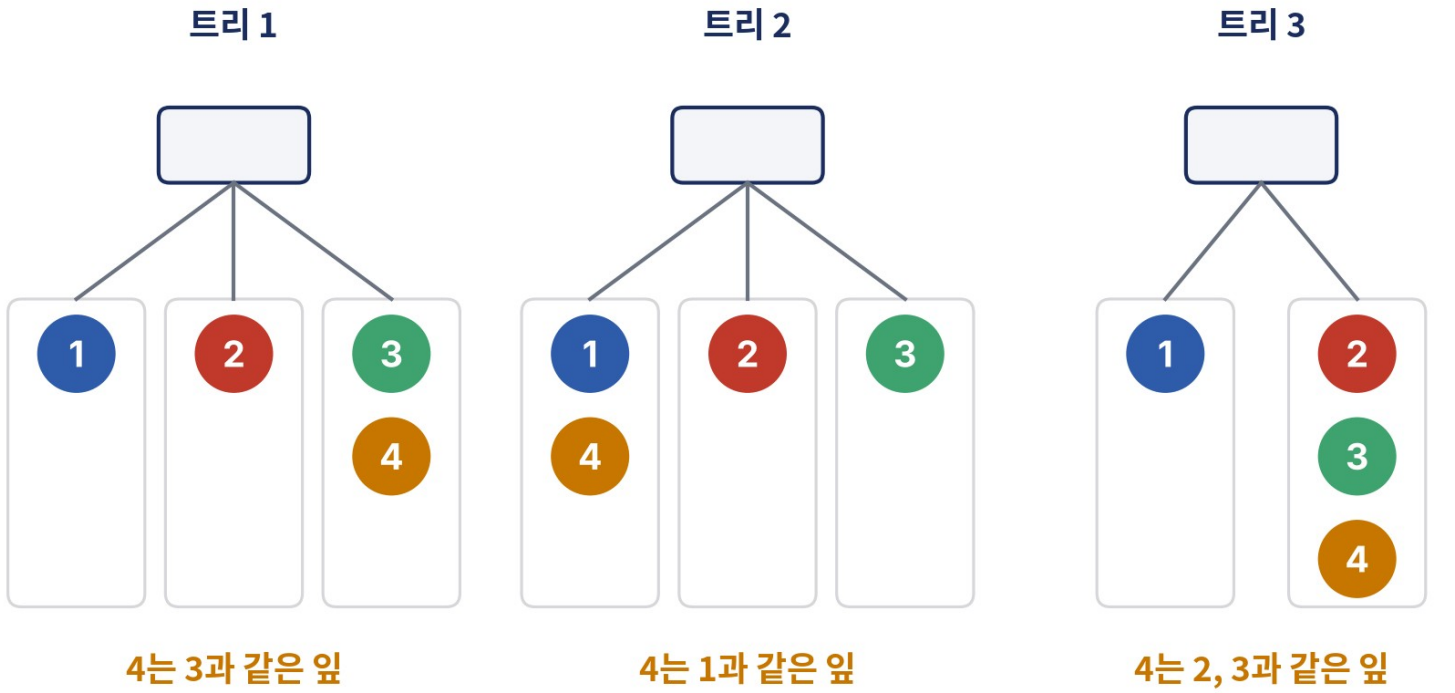
라벨이 같은 환자만 본다(심장병 없음: #1, #3)

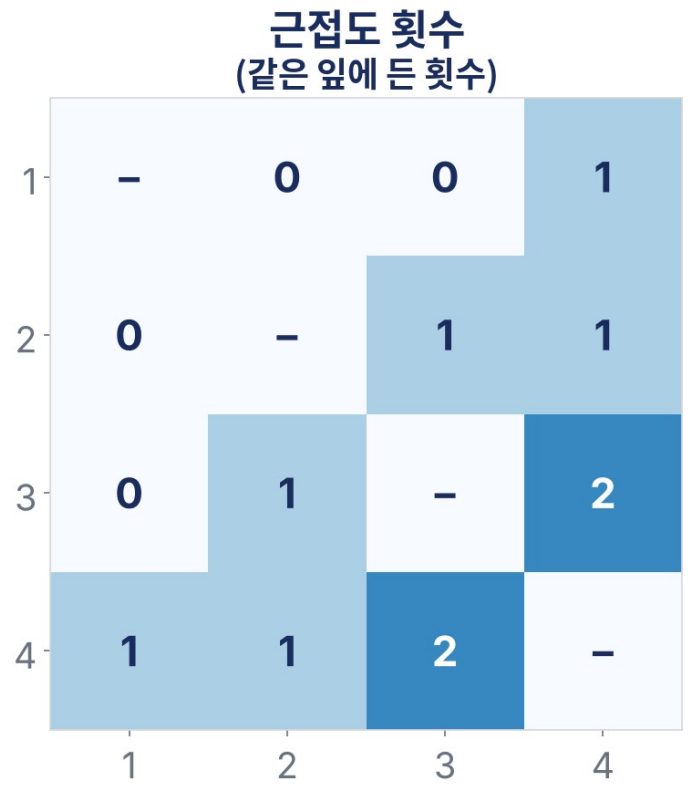
동맥 막힘 → 최빈값: 아니오

몸무게 → 125와 210의 중앙값: 167.5

2단계: 포레스트를 키우고 같은 잎에 들어간 횟수를 센다

근접도(proximity) = 두 환자가 같은 잎에 들어간 빈도. 그림은 트리 3개, 실제 포레스트는 수백 개

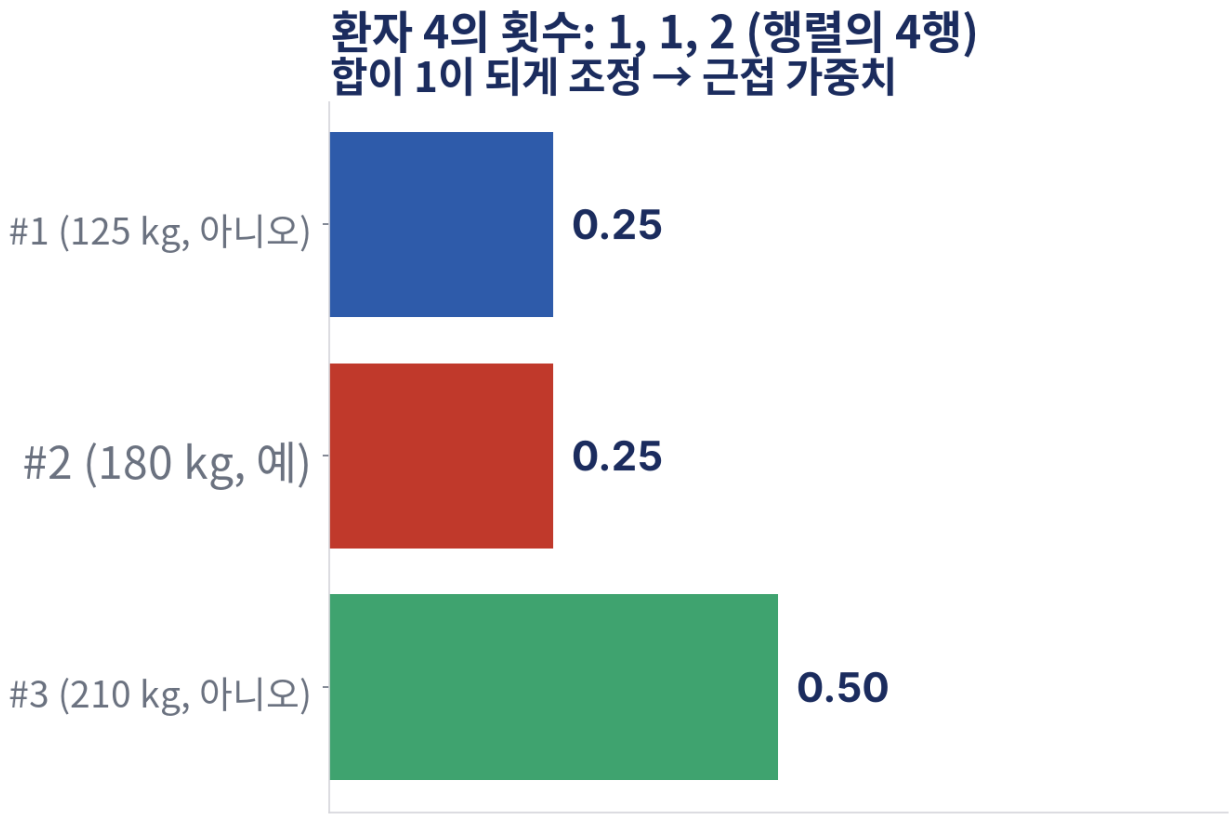




트리마다 같은 잎에 들어간 환자 쌍에 +1

3단계: 가장 가까운 이웃으로 빈칸을 다시 채운다

숫자는 가중평균, 범주는 가중 투표로 채운다



몸무게(숫자): 가중평균

0.25 × 125 + 0.25 × 180 + 0.50 × 210

= 181.3 kg (첫 추정은 167.5)

동맥 막힘(범주): 가중 투표

예 0.25 (환자 2)

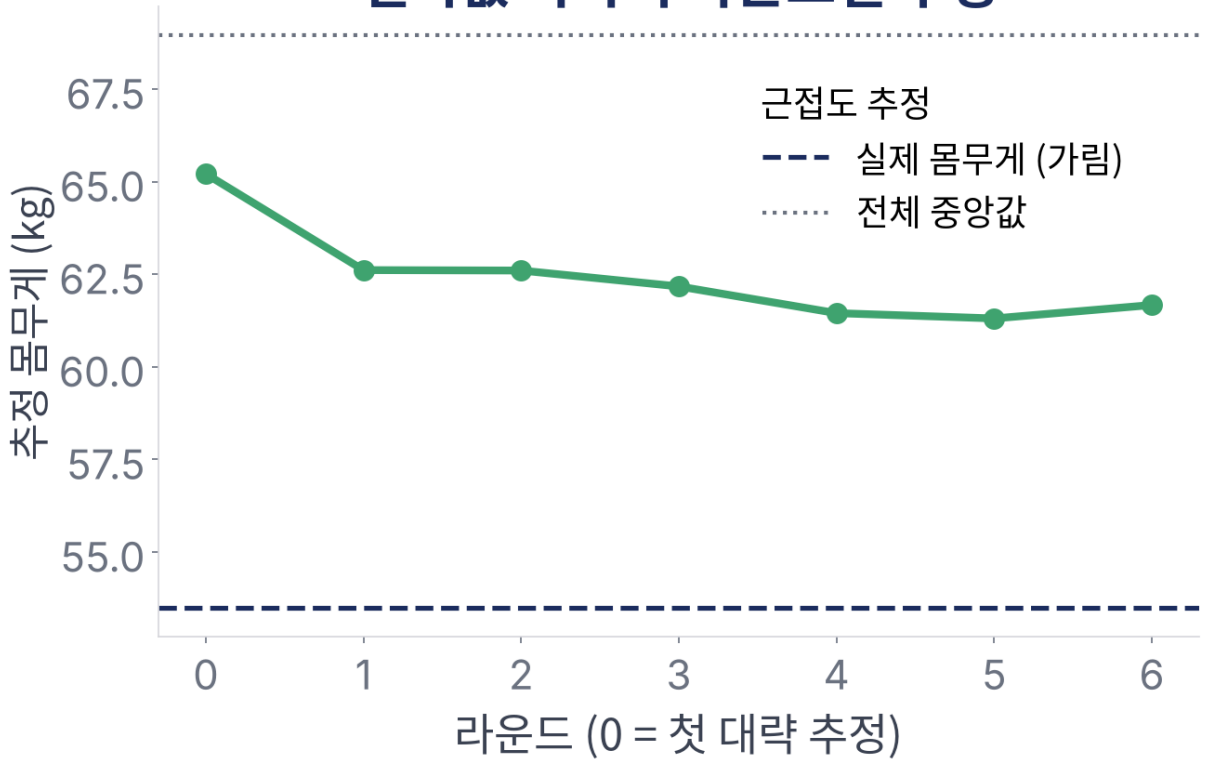
→ 아니오로 채움

아니오 0.75 (환자 1, 3)

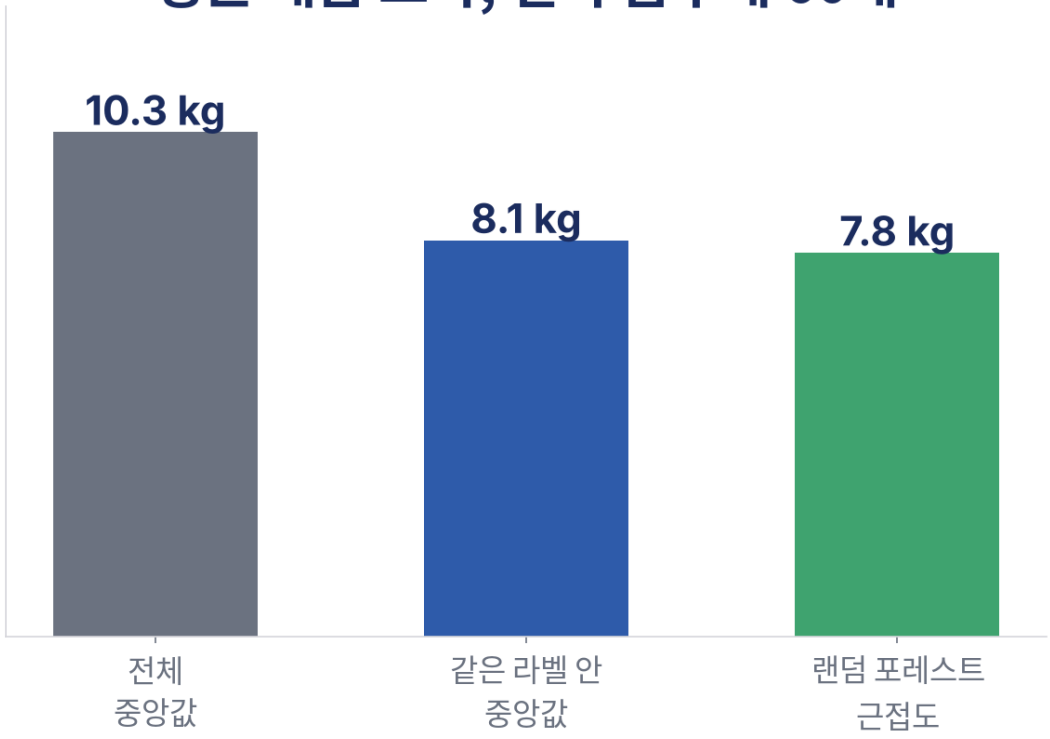
4단계: 추정값이 안정될 때까지 반복한다

시뮬레이션 데이터: 환자 300명, 몸무게 20% 가림, 라운드마다 포레스트를 다시 키움

결측값 하나의 라운드별 추정



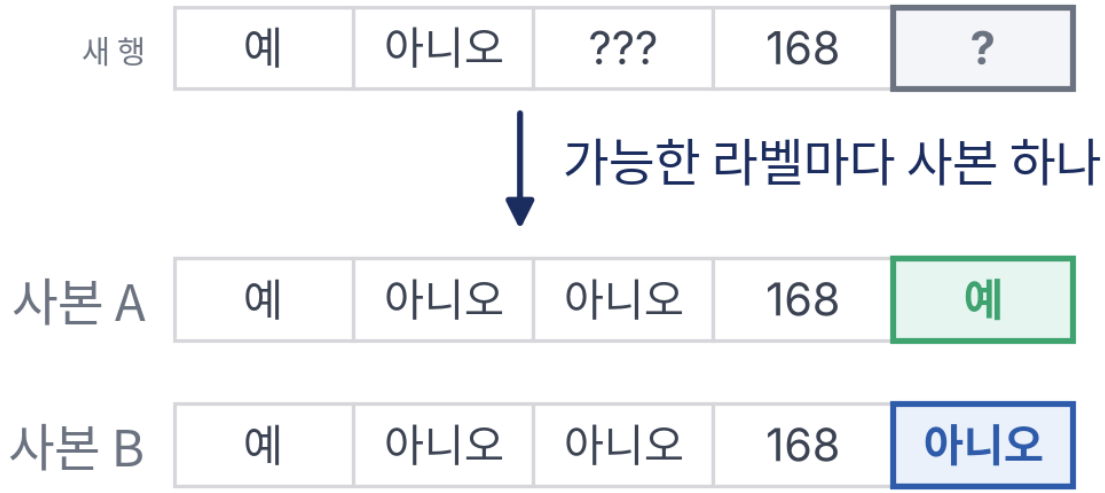
평균 채움 오차, 결측 몸무게 60개



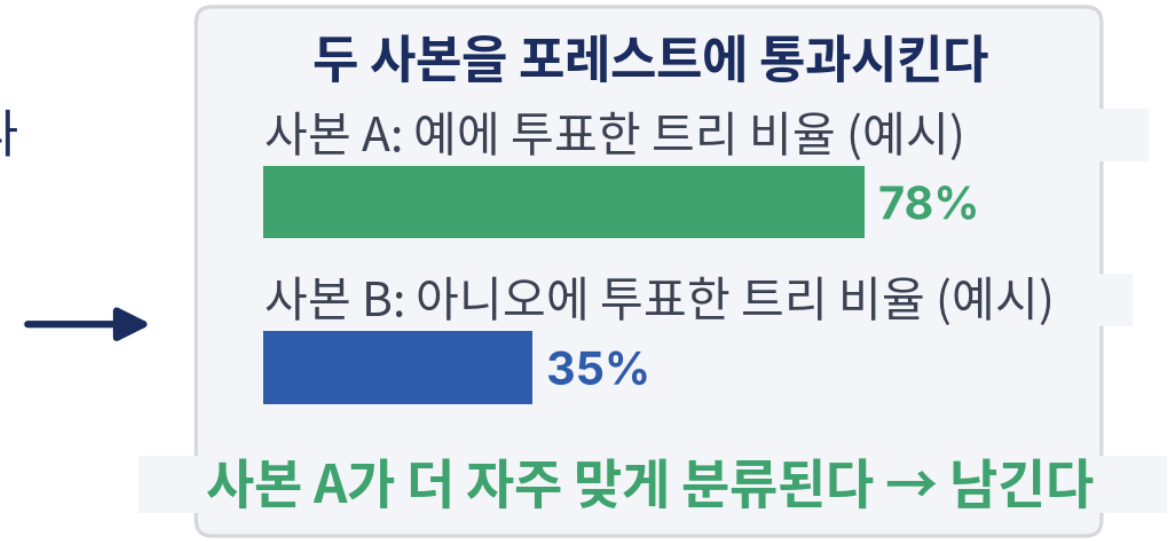
라벨이 없는 새 행은 사본을 만들어 처리한다

가능한 라벨마다 빈칸을 채운 사본을 만들고, 포레스트가 가장 자주 올바르게 분류한 사본을 남긴다

새 환자: 라벨을 모르고 값 하나가 결측이다



각 빈칸은 그 라벨의 대략 추정으로 채운다



근접도 채우기: 장점과 한계

언제 쓰고 무엇을 조심할까

비슷한 행이 값을 빌려준다. "비슷함"은 포레스트가 정한다

장점

- 숫자와 범주 모두에 쓸 수 있다.
- 한 열만이 아니라 다른 피처를 모두 활용한다.
- 거리 공식이나 스케일 조정이 필요 없다.

한계

- 느리다. 라운드마다 포레스트를 새로 키운다.
- 같은 라벨 중앙값 대비 개선 폭이 작을 수 있다.
- 도구: R의 rfImpute. MissForest가 가까운 방법이다.

03

3부 · 부스팅

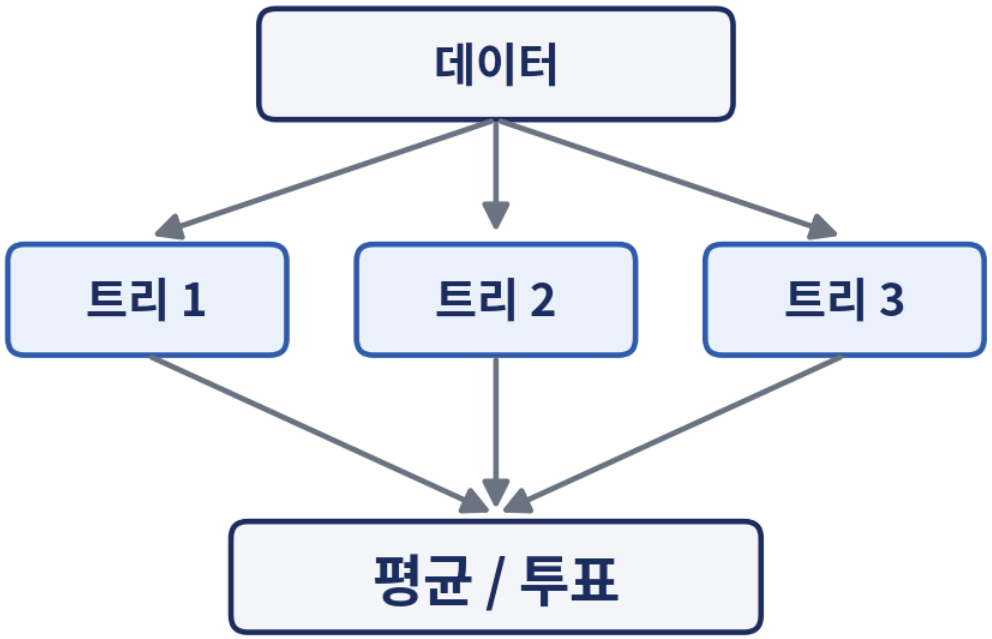
순차적으로 쌓는 작은 트리

AdaBoost와 그래디언트 부스팅: 각 트리가 앞선 실수에서 배운다

배깅은 트리를 평균하고 부스팅은 트리를 잇는다

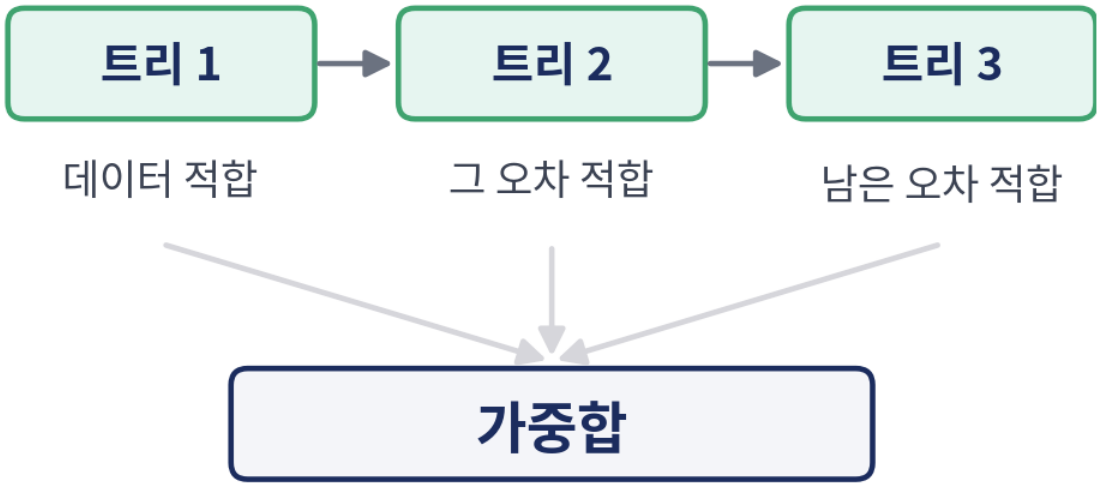
배깅은 분산을, 부스팅은 편향을 줄인다

배깅 / 랜덤 포레스트: 병렬



독립적인 깊은 트리 → 분산 감소

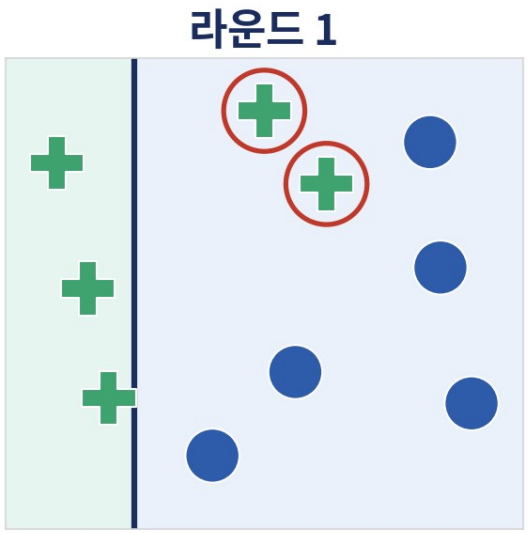
부스팅: 순차



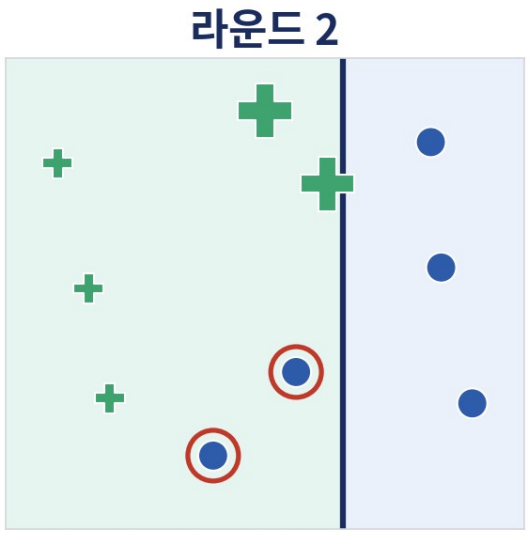
작은 트리가 서로 보완 → 편향 감소

AdaBoost는 틀린 점의 가중치를 키운다

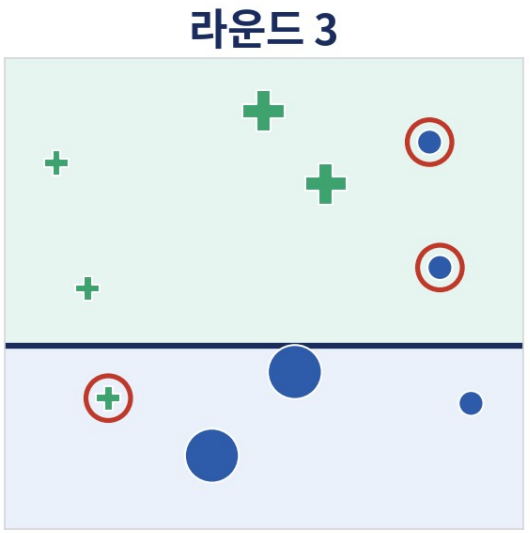
고전적 예시: 라운드마다 직선 하나로 자르고, 빨간 원은 틀린 점이다



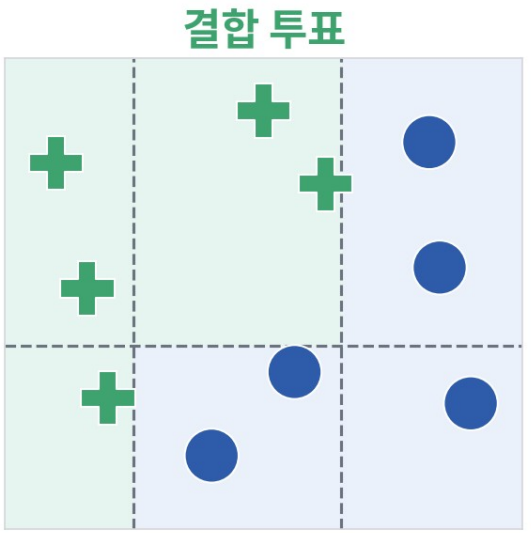
절단 하나, 실수 2개



라운드 1의 실수가
이제 더 크게 보인다



다음 절단은
큰 점에 집중한다



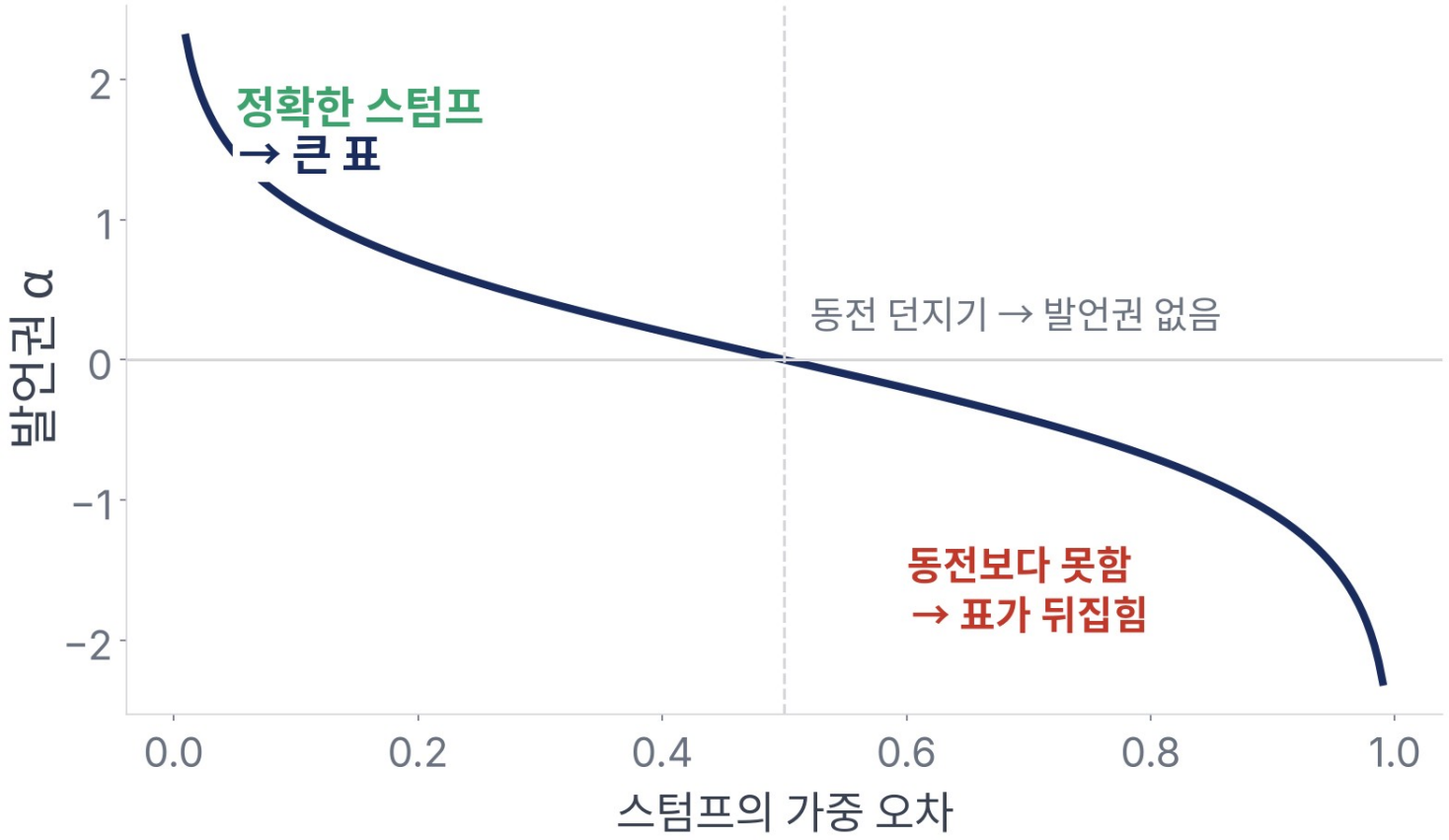
**약한 절단 셋을 합치면
모든 점을 분류한다**

정확한 절단일수록 큰 표를 받는다

절단마다 투표하며, 더 좋은 절단이 최종 결정에서 더 크게 반영된다

곡선 읽기

- 실수가 적다 → 큰 표.
- 동전 던지기 수준 → 표 없음.
- 동전보다 못하다 → 표가 뒤집힌다.



AdaBoost가 모든 카메라에 얼굴 검출을 넣었다

비올라·존스(Viola and Jones, 2001)는 부스팅한 스텀프를 빠른 캐스케이드로 이었다

단순한 명암 패턴 수천 개를 각각 스텀프(stump) 하나로 두고 AdaBoost로 결합했다

부스팅이 맞았던 이유

- 패턴 하나는 무작위 추측보다 겨우 낫다.
- 부스팅이 쓸모 있는 몇 개를 골라 가중치를 준다.
- 가벼운 앞 단계가 얼굴이 아닌 영역을 일찍 걸러 실시간으로 돈다.

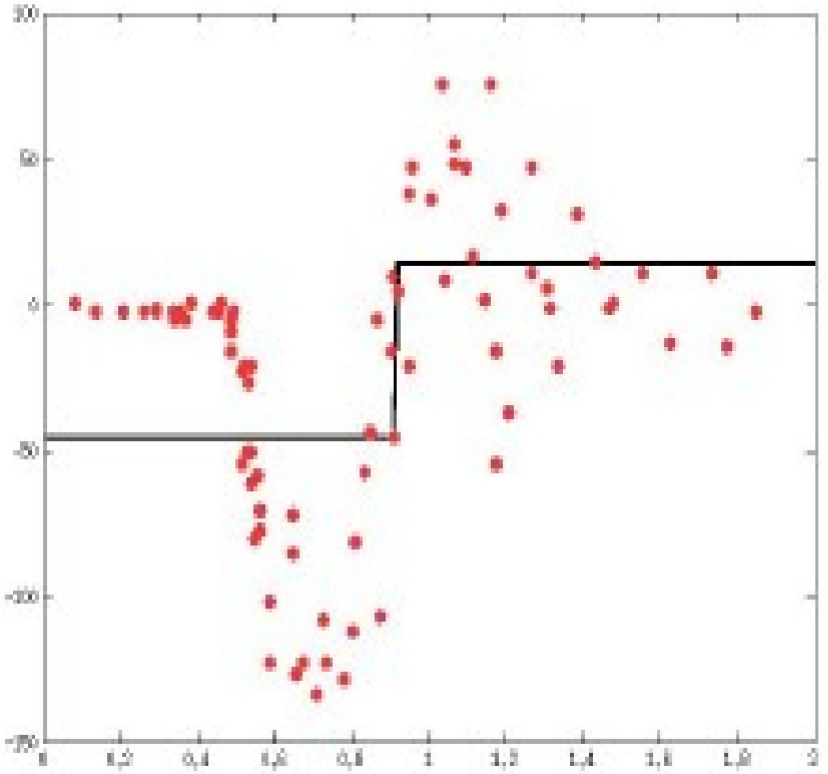
인정

- 프로인트와 샤피르(Freund and Schapire)는 AdaBoost로 2003년 괴델상을 받았다.
- 기저학습기(weak learner)를 강한 학습기로 바꾸는 발상이 그래디언트 부스팅으로 이어졌다.

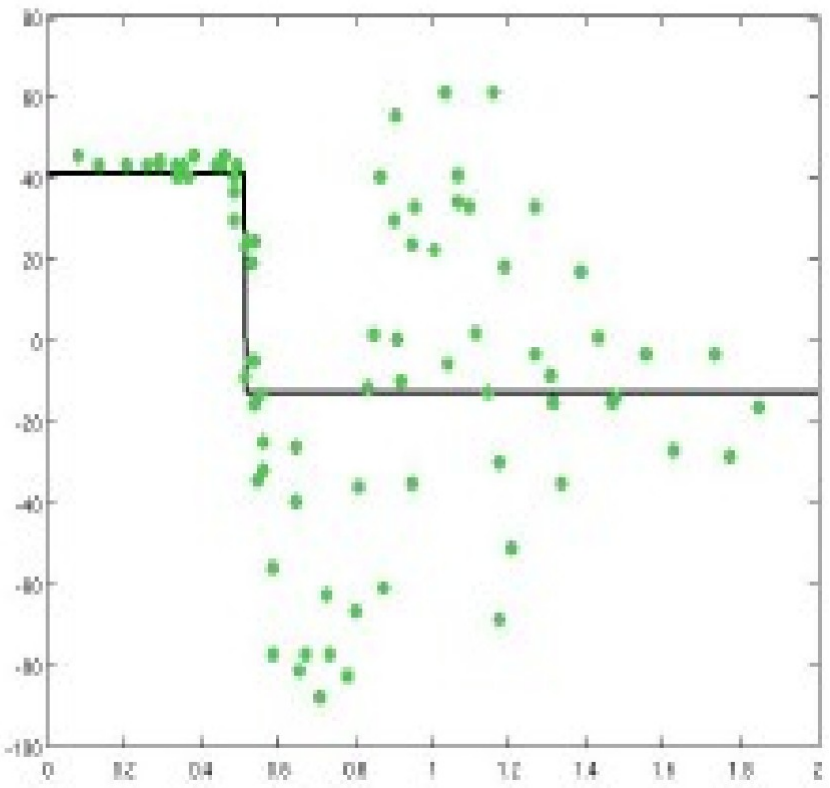
부스팅은 단순한 모형을 학습한 뒤 그 오차를 고친다

왼쪽: 첫 번째 작은 트리. 오른쪽: 첫 트리가 틀린 부분만으로 학습한 두 번째 트리

단순한 예측기를 학습하고…



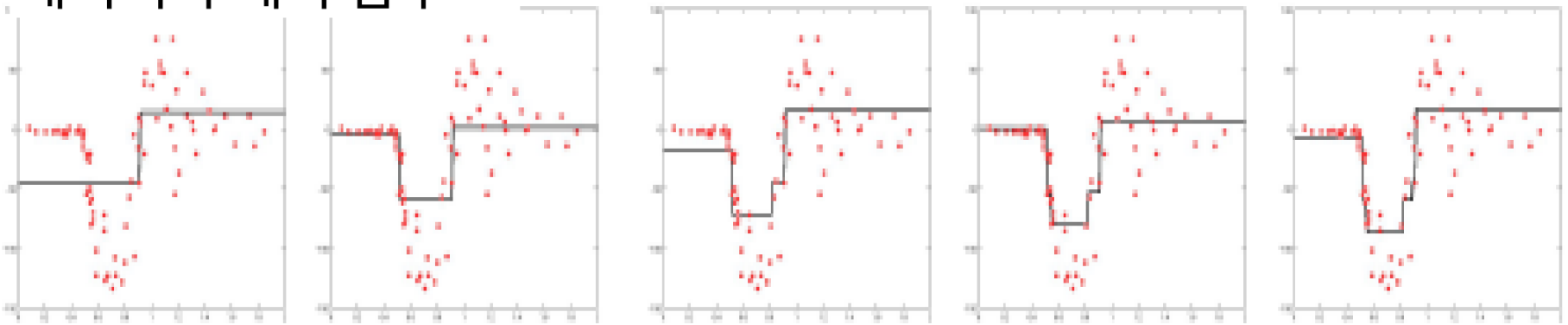
그 오차를 고쳐 나간다



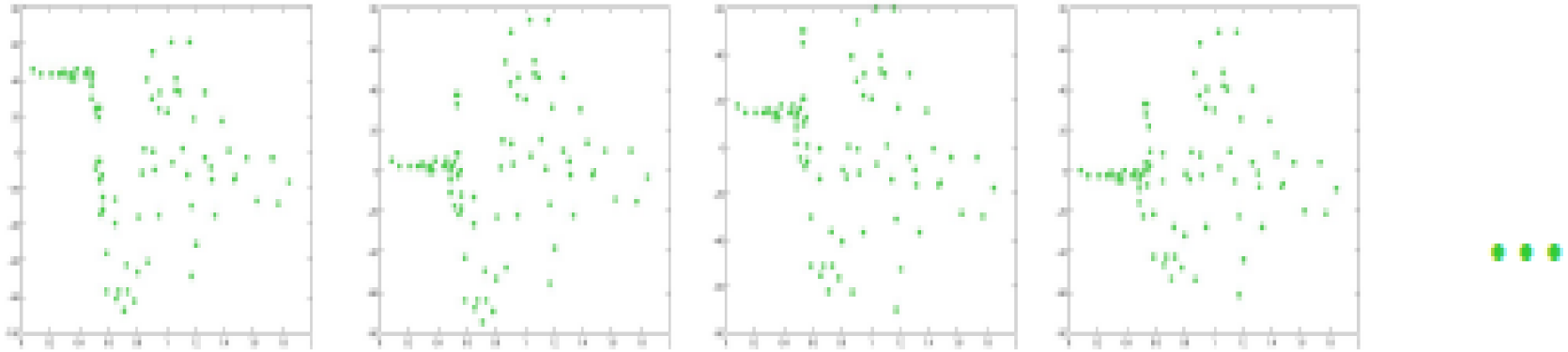
라운드마다 앞 라운드가 남긴 오차를 적합한다

위: 라운드별 데이터와 예측. 아래: 다음 트리가 학습할 남은 오차(잔차)

데이터와 예측 함수



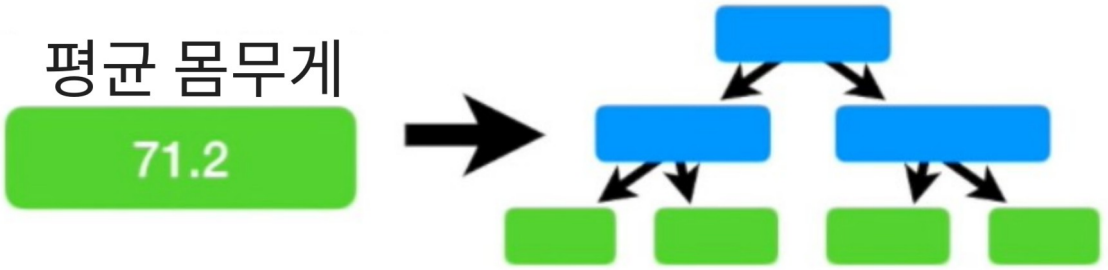
오차(잔차)



1단계: 평균 몸무게로 예측을 시작한다

키, 좋아하는 색, 성별이 있는 6명. 모두의 첫 추정은 평균 71.2 kg이다

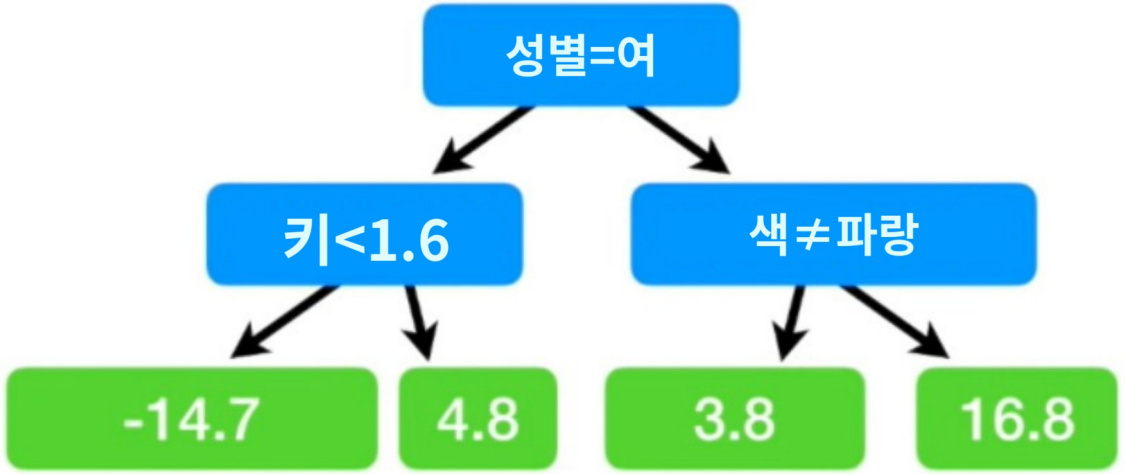
키 (m)	좋아하는 색	성별	몸무게 (kg)
1.6	파랑	남	88
1.6	초록	여	76
1.5	파랑	여	56
1.8	빨강	남	73
1.5	초록	남	77
1.4	파랑	여	57



2단계: 작은 트리가 잔차를 학습한다

잔차 = 관측 몸무게 − 71.2. 한 잎에 잔차가 둘이면 그 평균을 잎 값으로 쓴다

키 (m)	좋아하는 색	성별	몸무게 (kg)	잔차
1.6	파랑	남	88	16.8
1.6	초록	여	76	4.8
1.5	파랑	여	56	-15.2
1.8	빨강	남	73	1.8
1.5	초록	남	77	5.8
1.4	파랑	여	57	-14.2



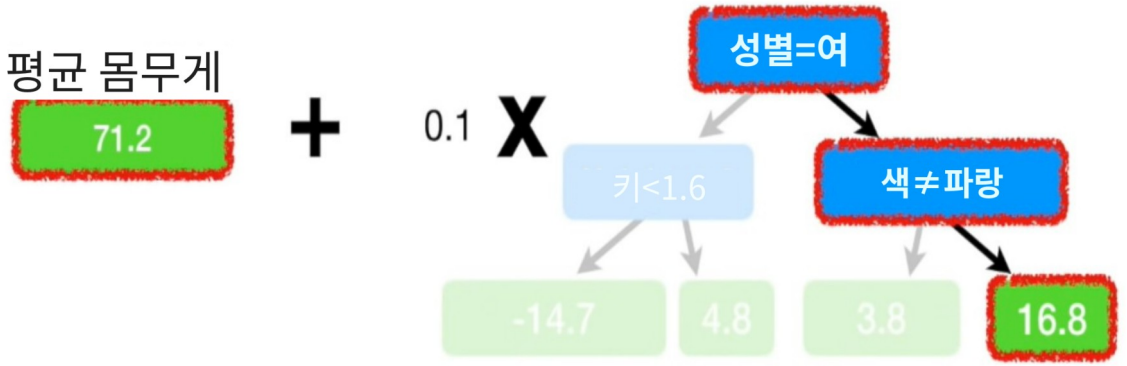
3단계: 한 번에 다 가면 외우고, 조금씩 가면 일반화한다

전체 보폭: 1번 사람의 예측이 정확히 88 kg. 학습률 0.1이면 72.9 kg에 그친다

예측 몸무게 = 71.2 + 16.8 = 88

키 (m)	좋아하는 색	성별	몸무게 (kg)
1.6	파랑	남	88

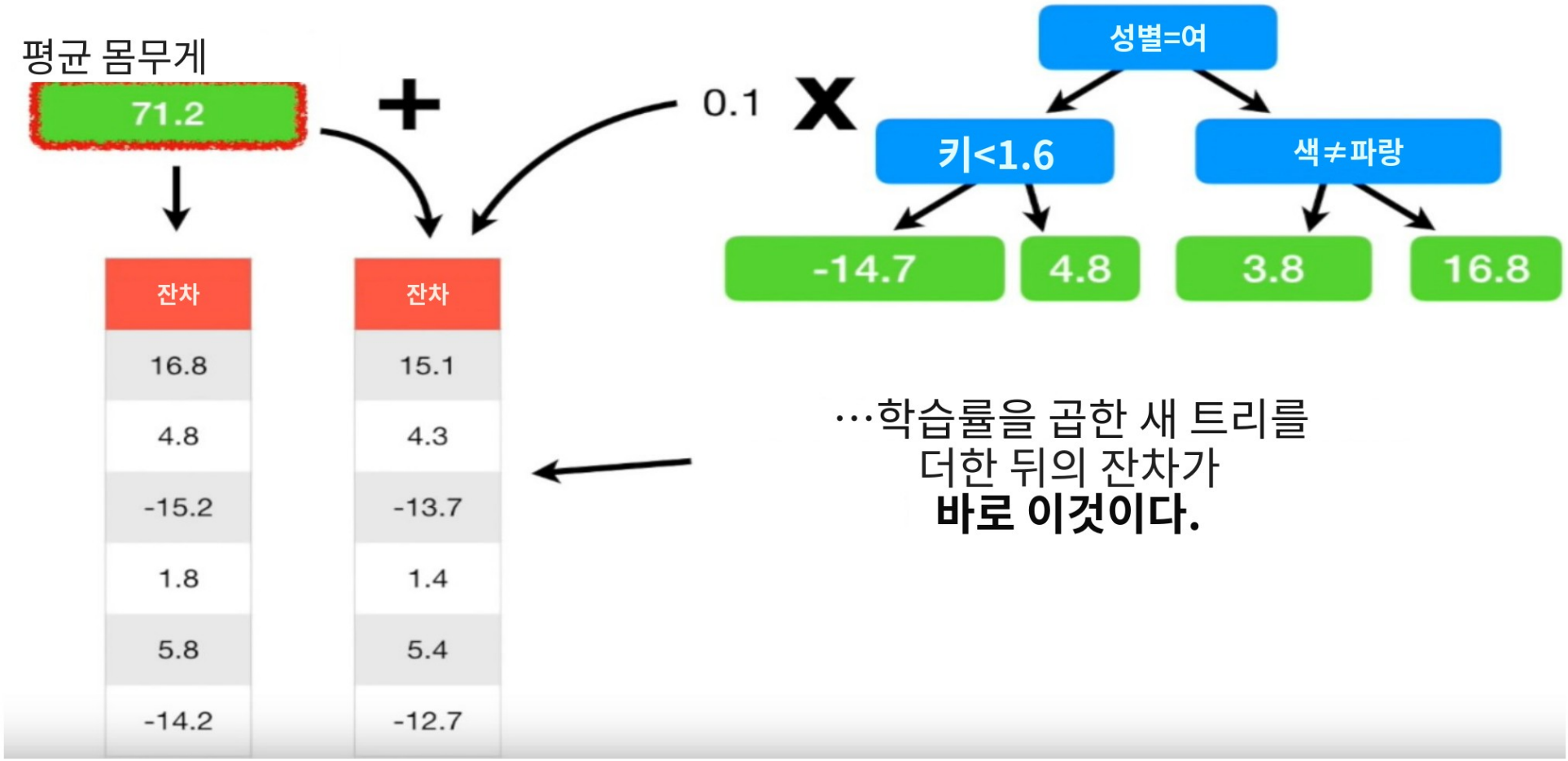
…관측 몸무게와 **같다.**



이제 예측 몸무게 = 71.2 + (0.1 × 16.8) = 72.9

4단계: 작은 보폭 한 번 뒤 잔차가 줄어든다

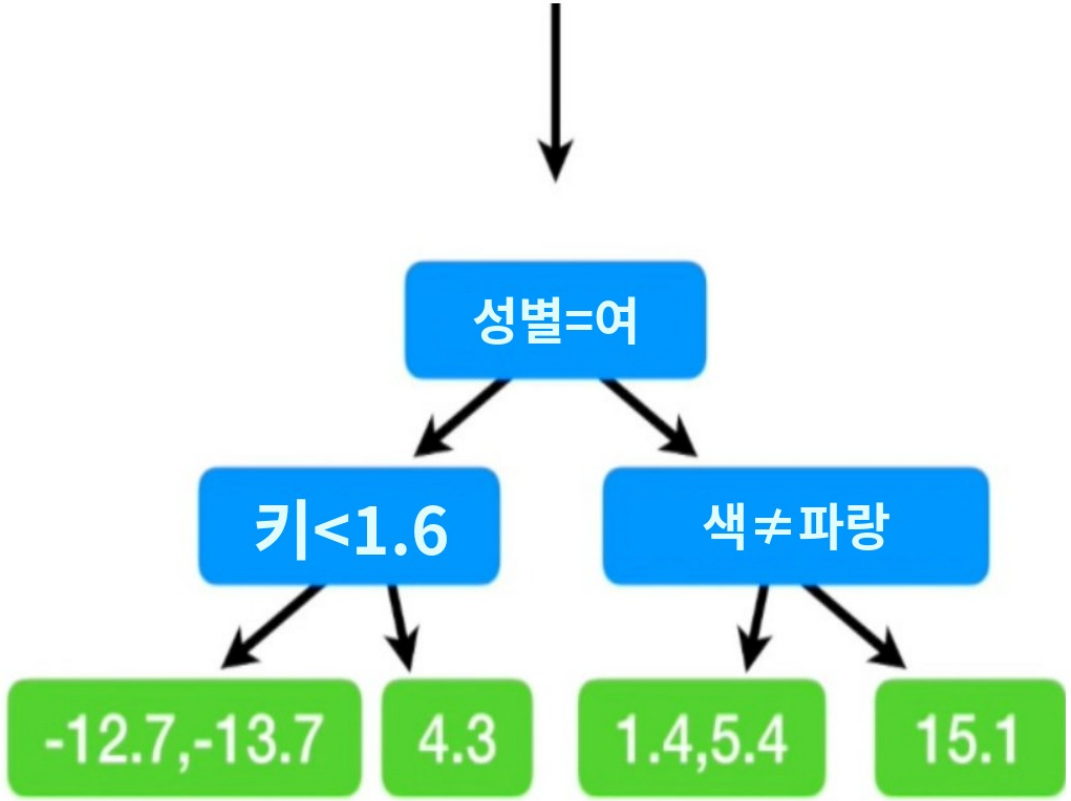
왼쪽 열: 첫 잔차(평균 기준). 오른쪽 열: 0.1 × 트리 1을 더한 뒤의 잔차



5단계: 두 번째 트리가 새 잔차를 학습한다

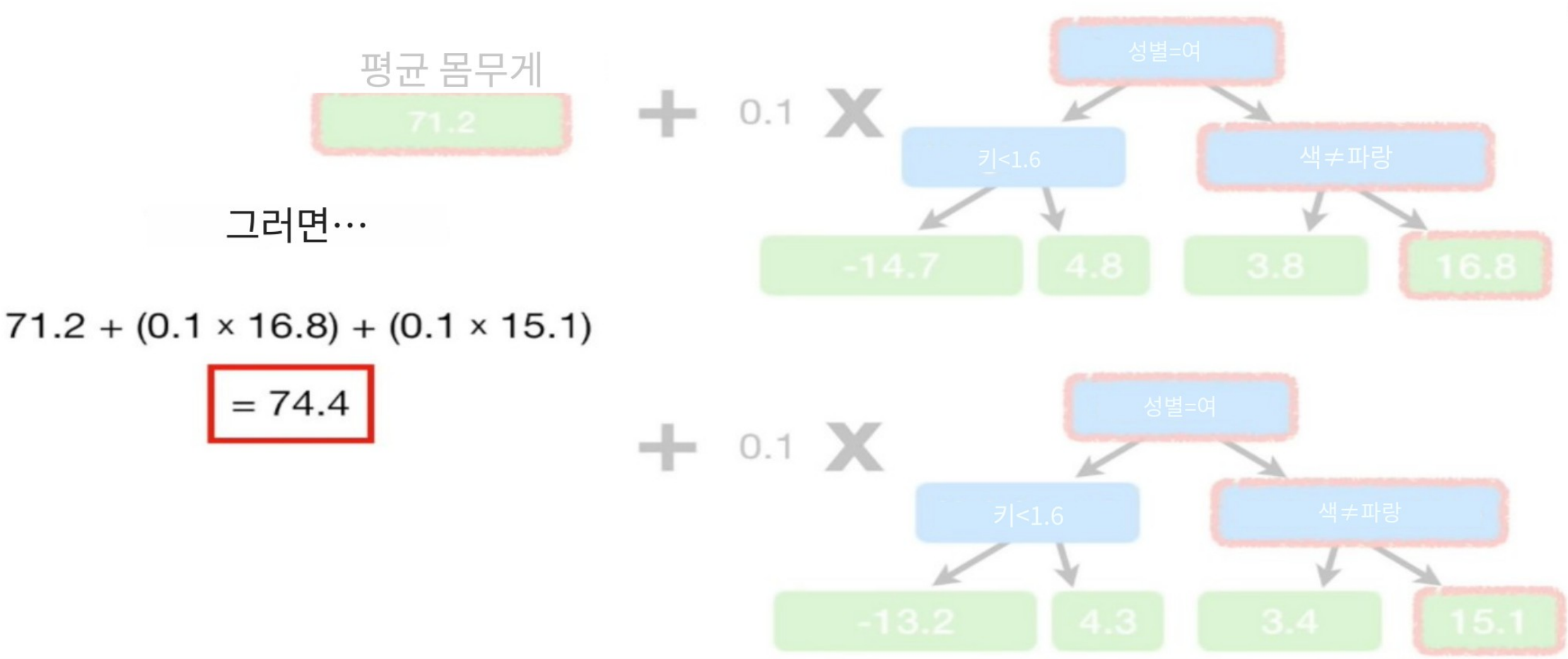
피처는 같고 목표값이 새롭다. 행이 둘인 잎은 다시 평균을 쓴다

키 (m)	좋아하는 색	성별	몸무게 (kg)	잔차
1.6	파랑	남	88	15.1
1.6	초록	여	76	4.3
1.5	파랑	여	56	-13.7
1.8	빨강	남	73	1.4
1.5	초록	남	77	5.4
1.4	파랑	여	57	-12.7



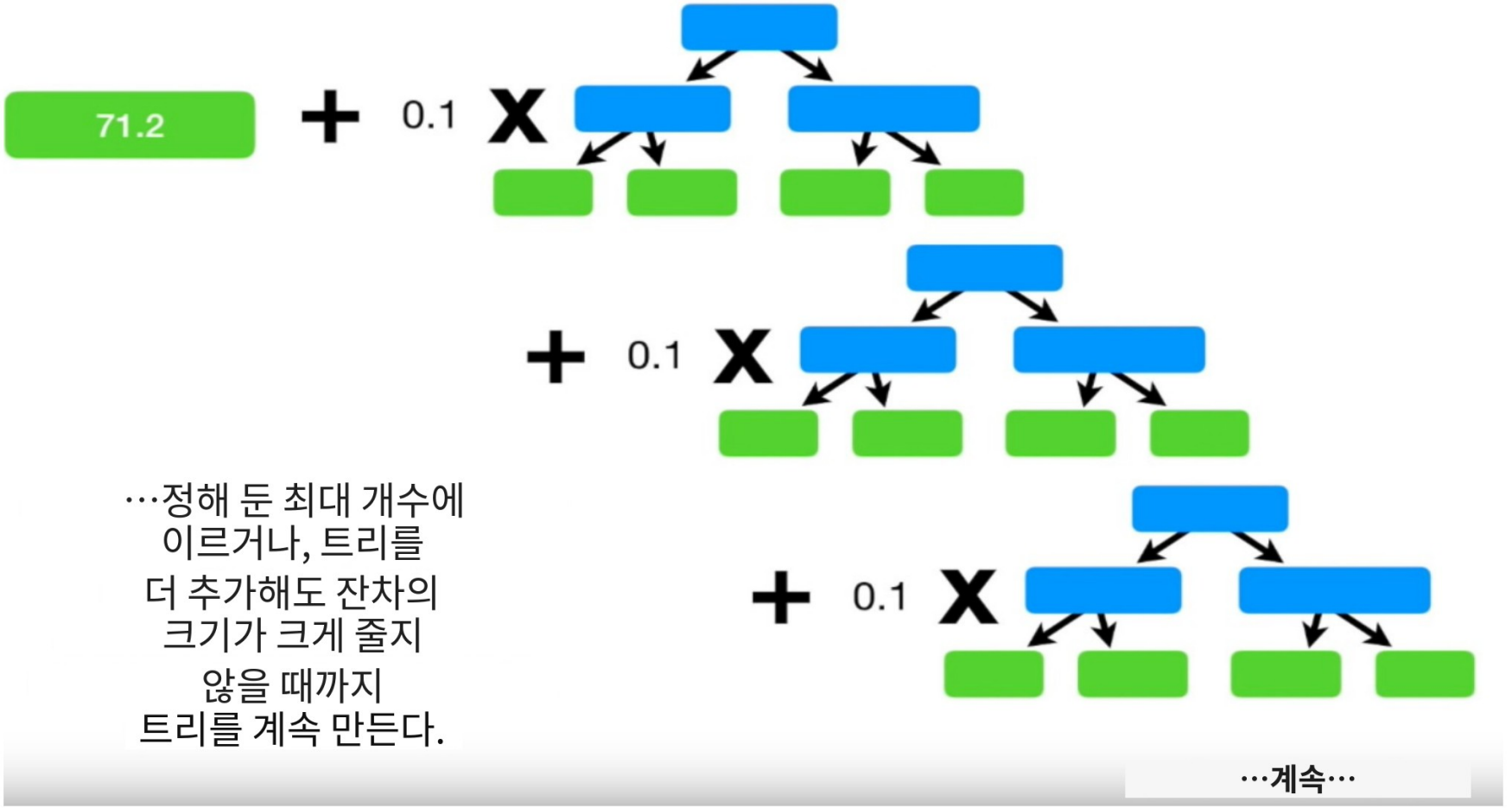
6단계: 작은 트리 두 개가 추정을 더 가깝게 옮긴다

1번 사람: 71.2 + 0.1 × 16.8 + 0.1 × 15.1 = 74.4 kg. 실제 몸무게는 88 kg



7단계: 작은 보정을 계속 더한다

정해 둔 트리 수에 이르거나, 트리를 더해도 잔차가 줄지 않으면 멈춘다

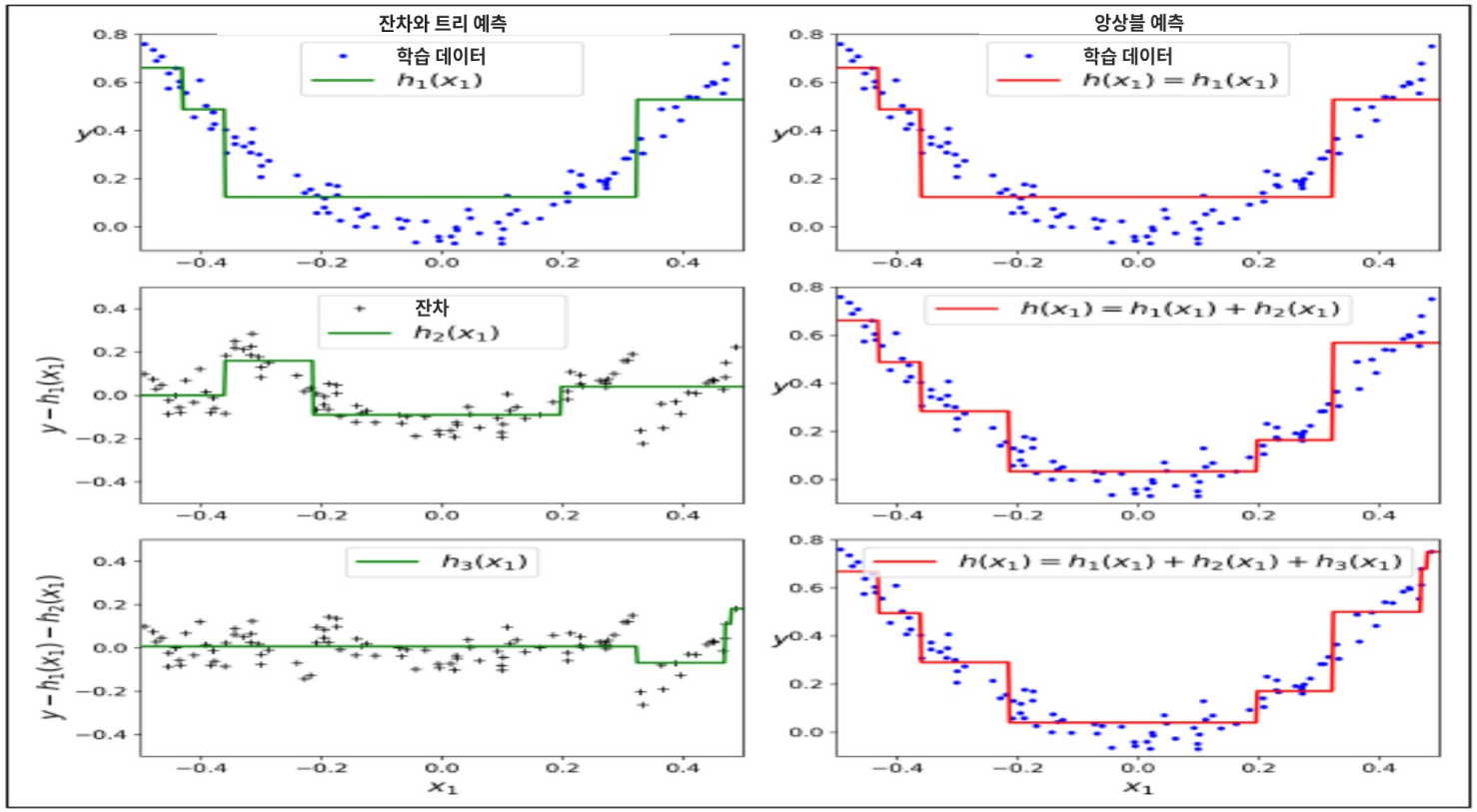


앙상블은 작은 트리 모두의 합이다

scikit-learn의 깊이 2 트리 세 개. 왼쪽: 트리별 적합. 오른쪽: 누적 합

읽는 법

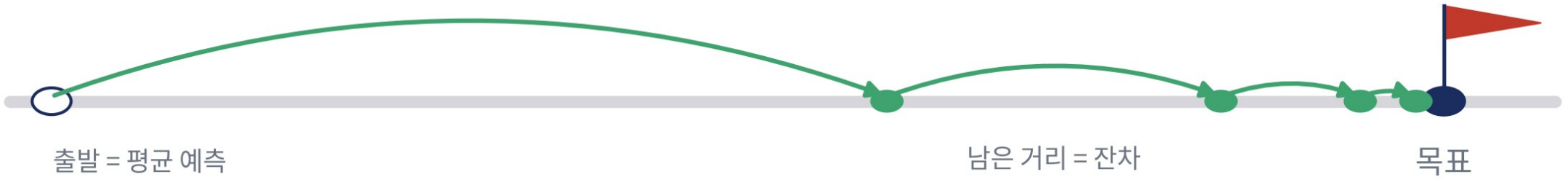
- 1행: 트리 1이 데이터를 적합한다.
- 2행: 트리 2가 트리 1의 오차를 적합한다.
- 3행: 트리 3이 아직 남은 부분을 적합한다.
- 오른쪽 열: 행이 내려갈수록 예측이 가까워진다.



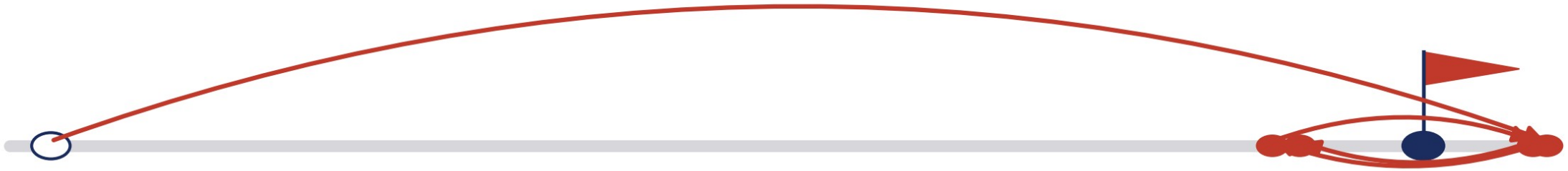
부스팅은 골프 퍼팅과 같다

트리 하나가 목표를 향한 한 번의 샷이고, 학습률은 얼마나 세게 칠지 정한다

작고 신중한 샷: 트리마다 남은 거리의 일부를 간다



전력 샷: 공이 계속 홀을 지나친다

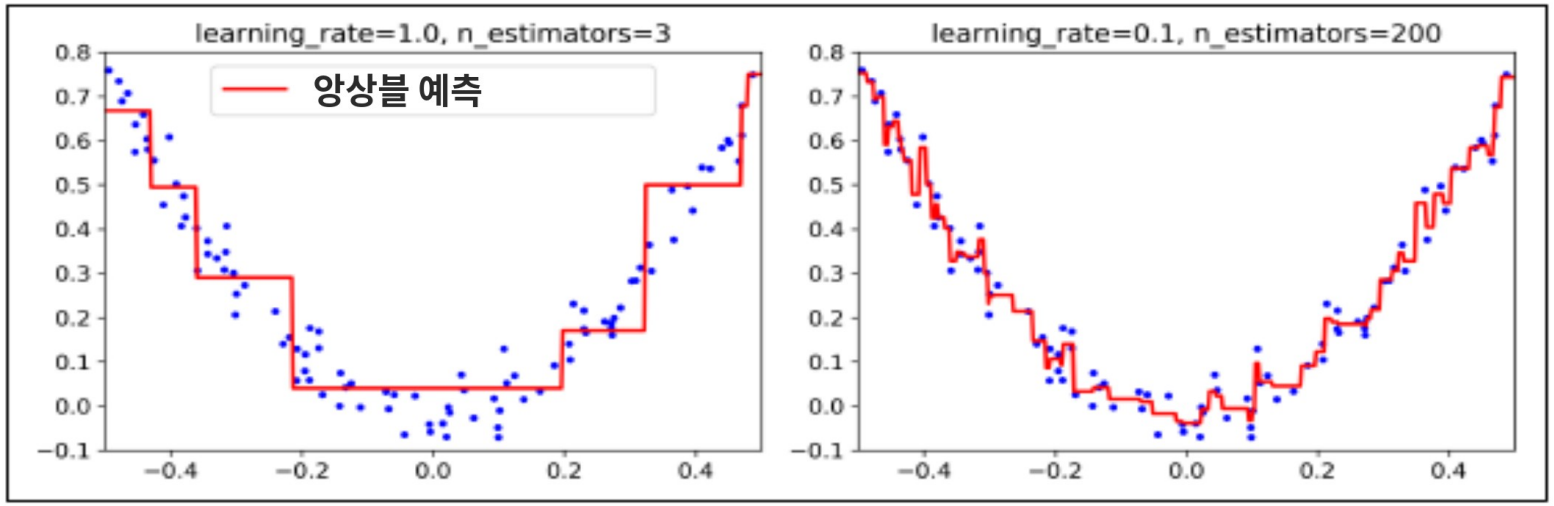


보폭이 작으면 트리가 더 필요하지만 더 잘 맞는다

왼쪽: 학습률 1.0, 트리 3개. 오른쪽: 학습률 0.1, 트리 200개

실무 규칙

- 큰 보폭: 트리가 적고 적합이 거칠다.
- 작은 보폭: 트리가 많고 적합이 매끄럽다.
- 학습률을 작게 두고 검증 오차가 더 줄지 않으면 멈춘다.



04

4부 · 최신 GBDT

XGBoost, LightGBM, CatBoost

부스팅 아이디어는 같고, 공학적 구현과 규제가 다르다

XGBoost는 아이디어는 그대로 두고 엔진을 개선한다

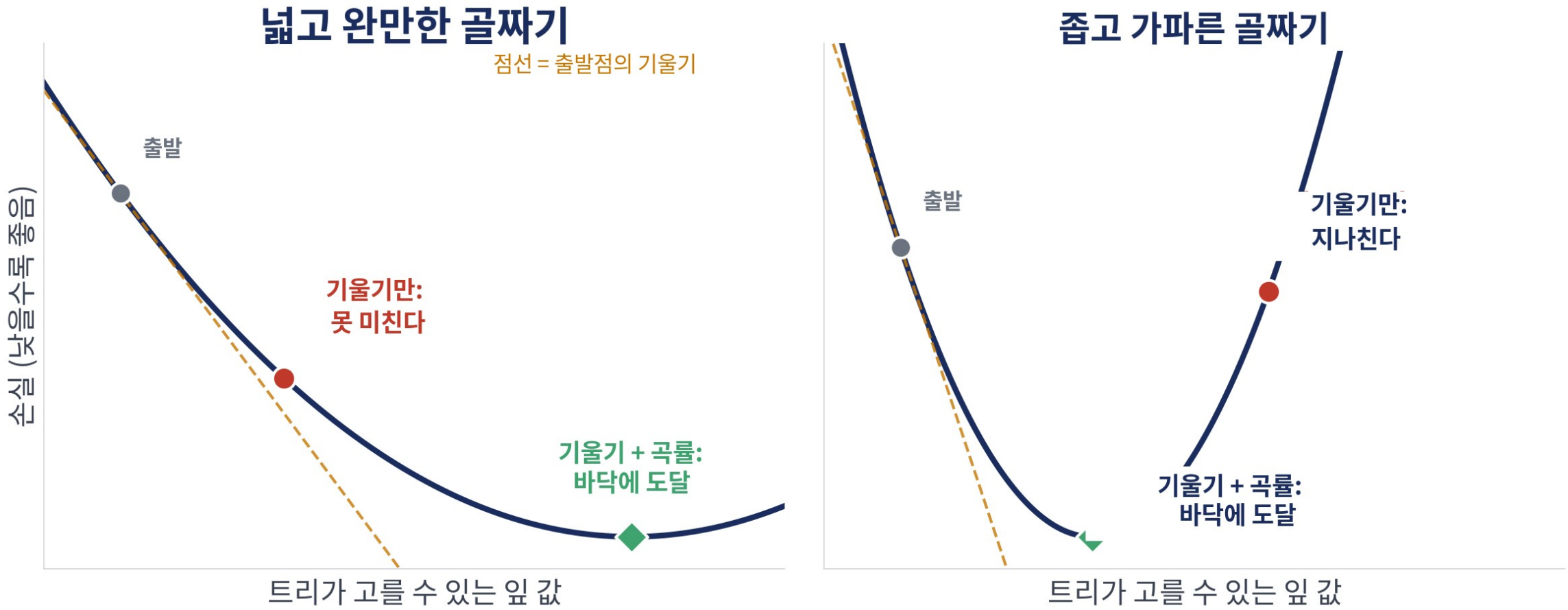
둘 다 앞선 오차를 고치는 작은 트리를 더한다. 차이는 그 방법에 있다

질문	고전적 그래디언트 부스팅	XGBoost
트리마다 얼마나 이동하나?	기울기만 쓴다	기울기와 곡률을 쓴다
트리 크기를 어떻게 작게 유지하나?	깊이 제한만	모든 분할에 벌점
결측값은?	먼저 채운다	보낼 방향을 학습한다
어떤 절단을 시험하나?	값 사이 간격 전부	구간 경계만
CPU 코어는 몇 개?	하나	전부

다음 슬라이드에서 차이를 하나씩 그림으로 본다

XGBoost는 기울기만이 아니라 곡률까지 본다

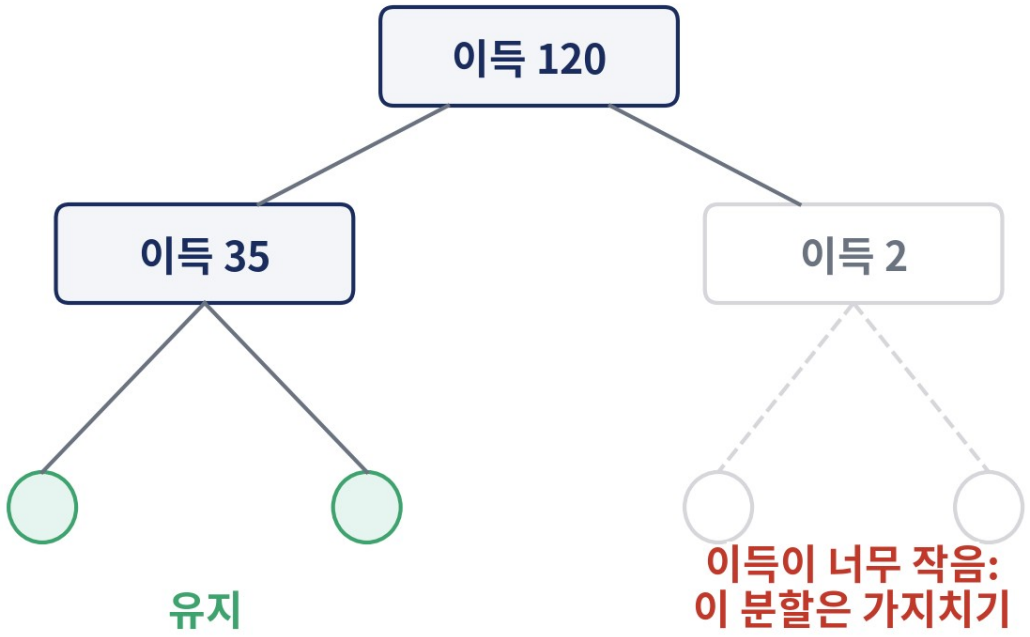
안개 속에서 비탈을 내려가는 것과 같다. 기울기는 방향을, 곡률은 바닥까지의 거리를 알려 준다



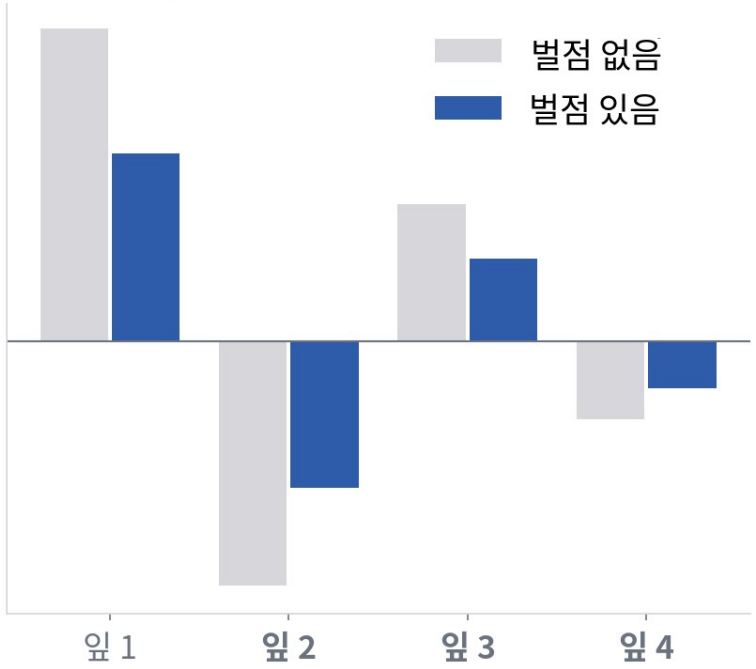
XGBoost는 트리를 작게 유지하는 벌점을 더한다

약한 분할은 가지치기하고 잎 출력은 줄여서, 트리마다 조금씩만 움직인다

벌점 1: 분할은 이득이 충분해야 한다

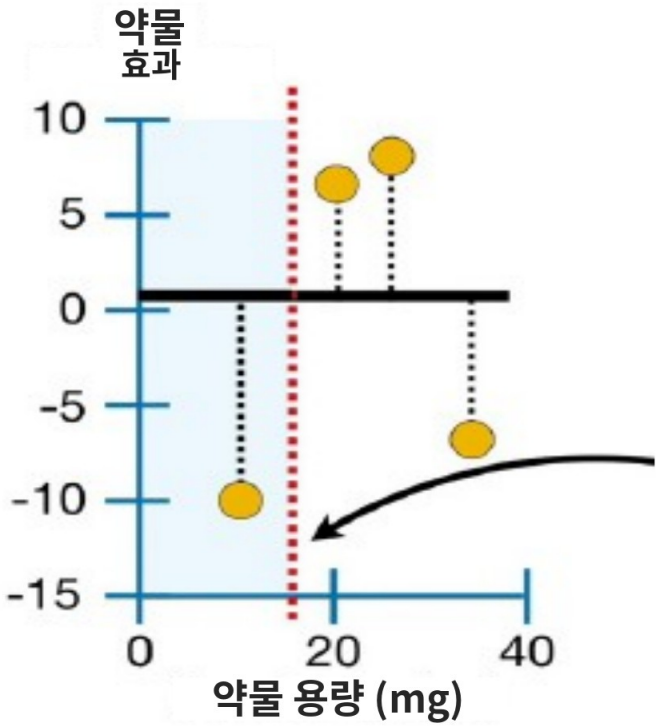


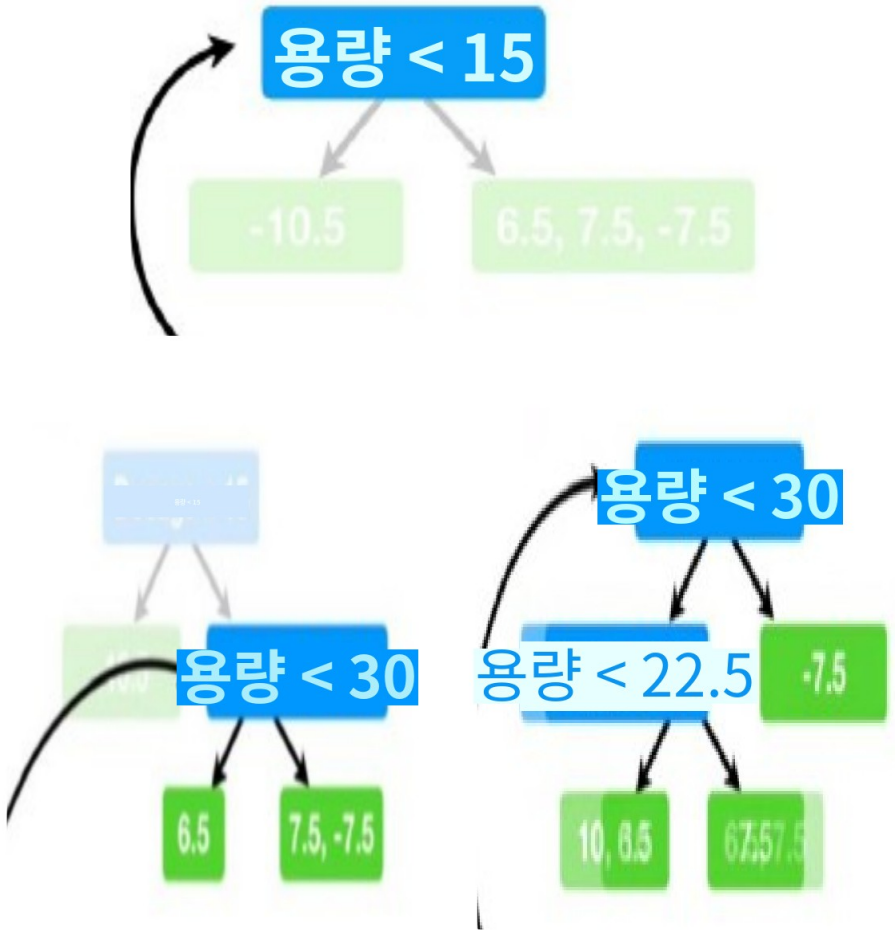
벌점 2: 잎 값을 0 쪽으로 당긴다



XGBoost는 이득이 가장 큰 절단을 고른다

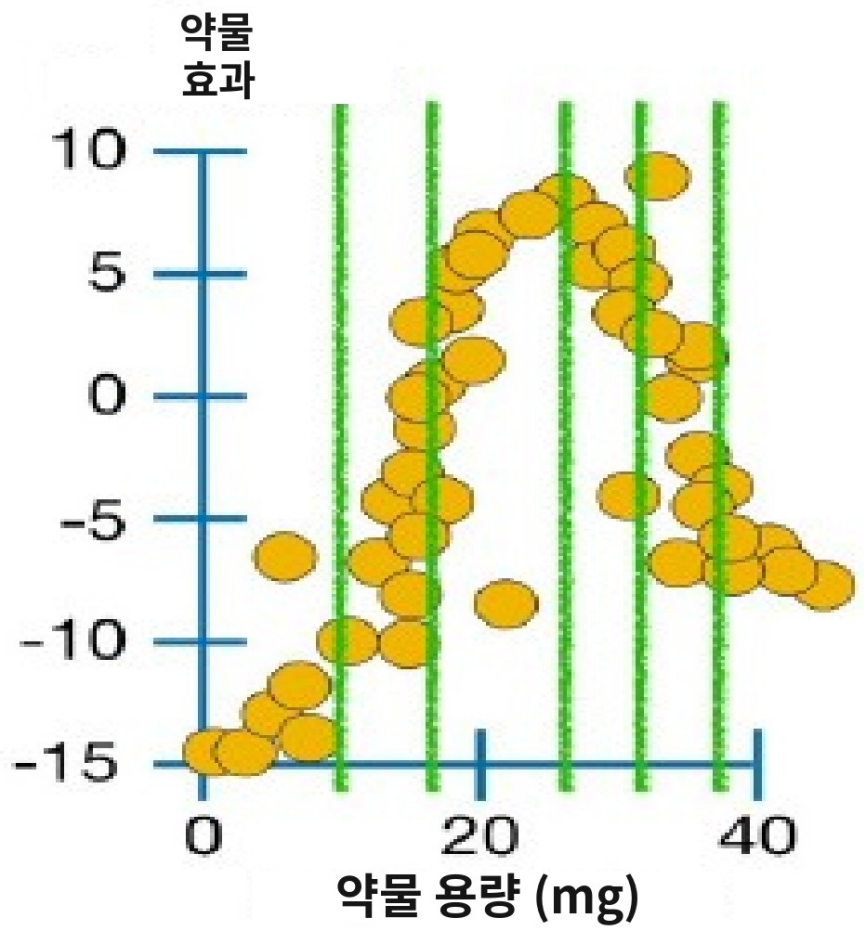
약물 용량 예시: 후보 절단마다 점수를 매기고, 가장 좋은 것을 분할로 쓴다

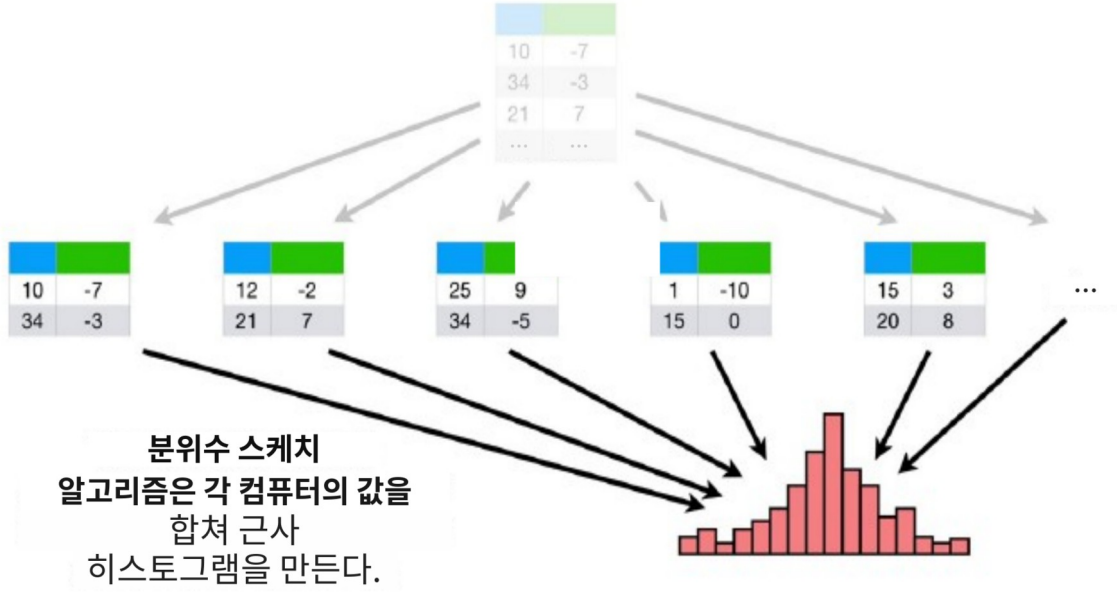




큰 데이터에서 XGBoost는 분위수 절단점만 시험한다

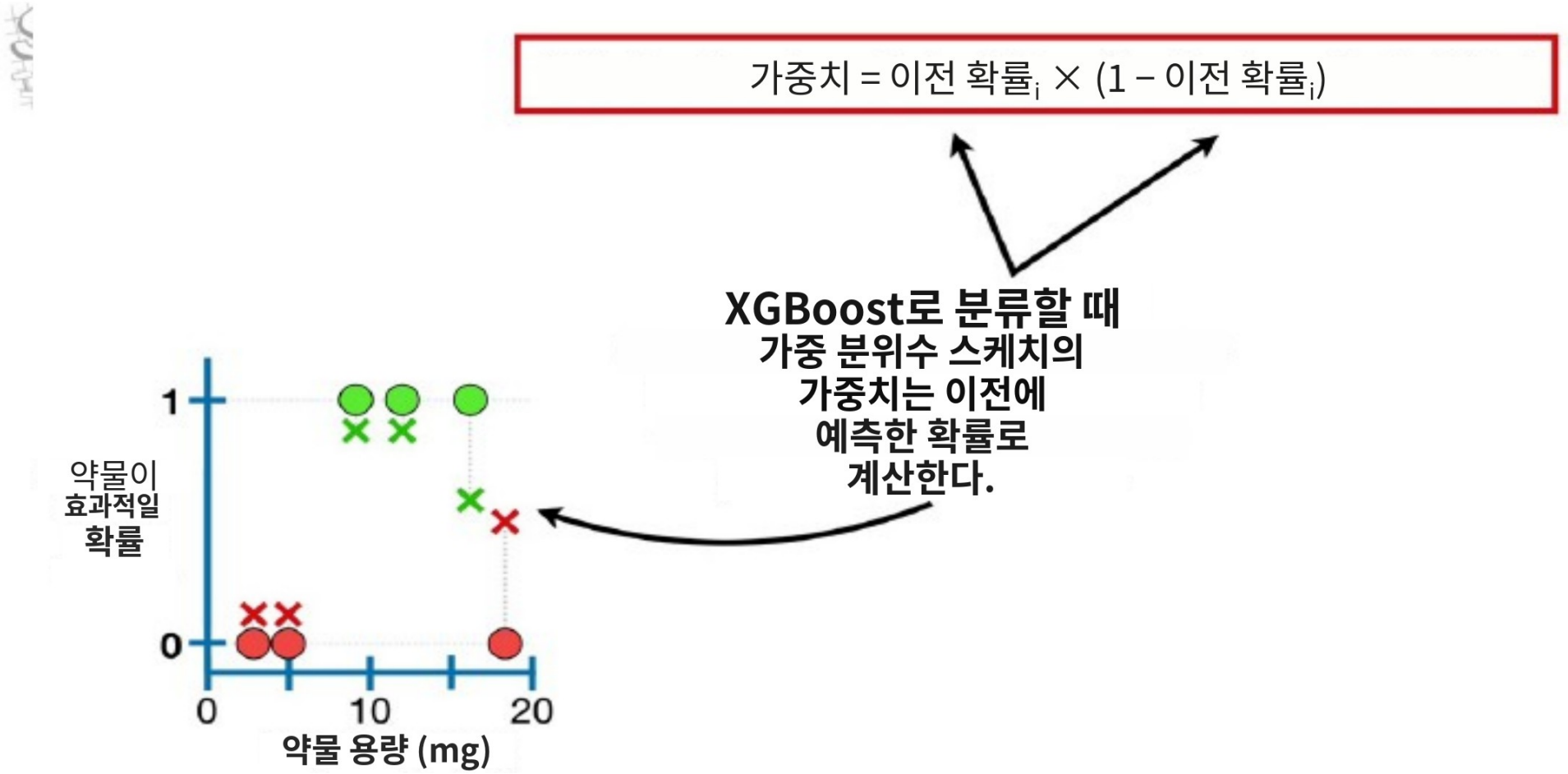
왼쪽: 분위수에서만 절단. 오른쪽: 여러 컴퓨터의 요약을 하나의 히스토그램으로 합친다





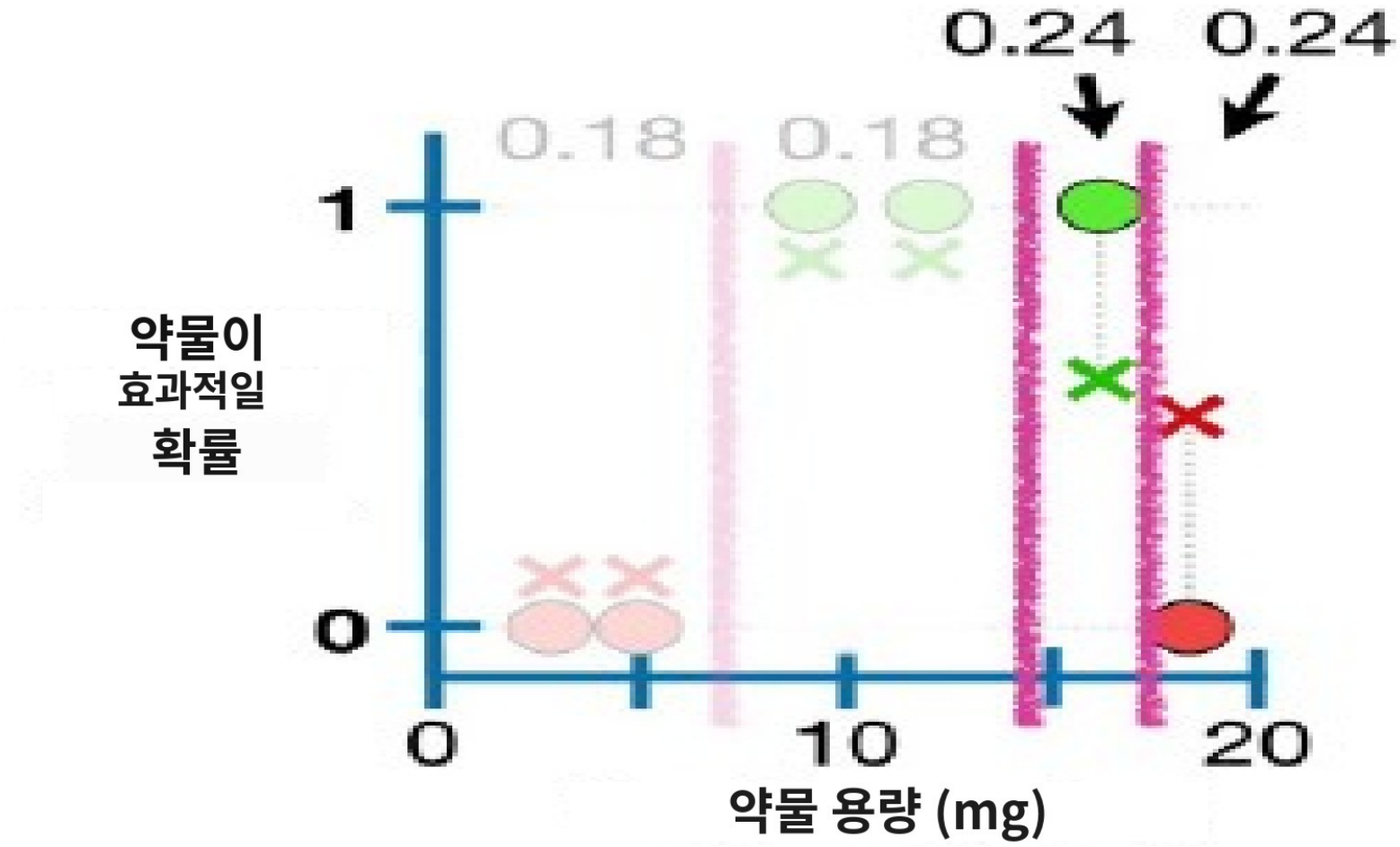
가중 분위수는 불확실한 행에 집중한다

각 행의 가중치는 p × (1 – p): 확신하는 행은 가볍고 불확실한 행은 무겁다



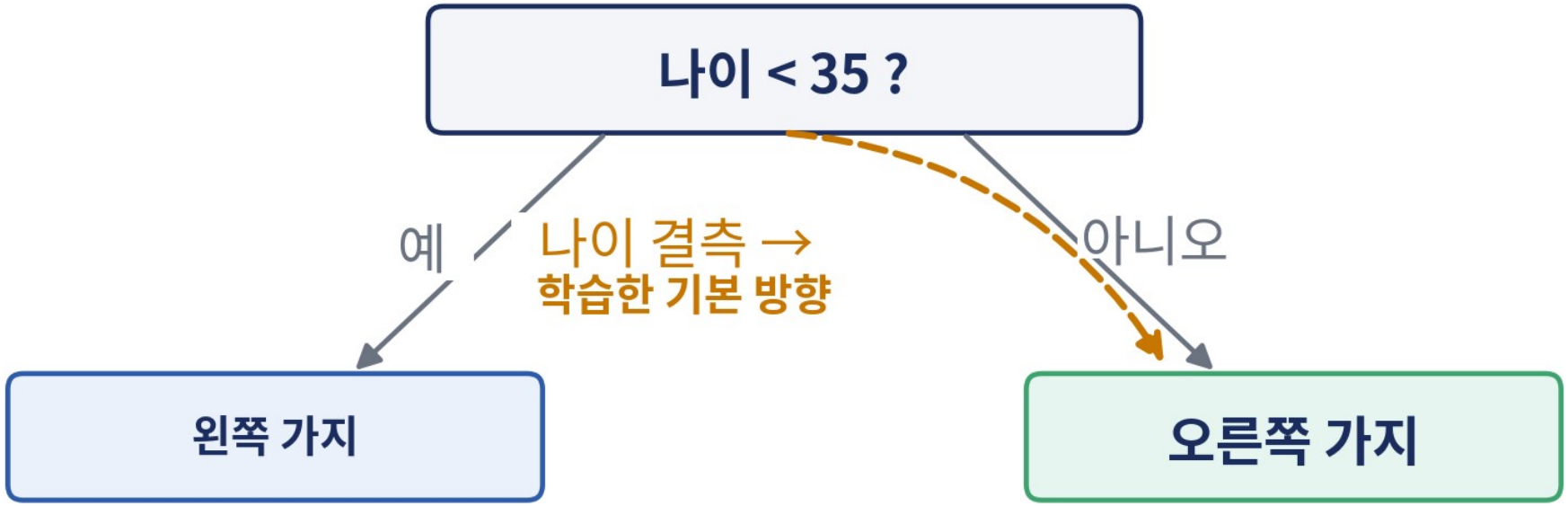
가중 분위수 구간마다 가중치 합이 같다

불확실한 행은 가중치가 0.24씩이라 한 행만으로 분위수 구간 하나를 채운다



결측값은 학습한 기본 방향을 따른다

희소성 인식 분할 탐색(sparsity-aware split finding): 학습 전에 결측값을 대체할 필요가 없다



XGBoost는 결측 행을 모두 왼쪽, 이어서 오른쪽으로 보내 보고 이득이 큰 쪽을 택한다

결측값이 있는 행에도 잔차는 있다

두 행은 용량이 결측(???)이지만 잔차는 잎에 넣을 수 있다



예측 약물
효과

0.5

용량	약물 효과	잔차
10	-7	-7.5
???	-3	-3.5
21	7	6.5
25	8	7.5
5	-5	-5.5
???	-2	-2.5

예측 약물
효과

0.5

용량	약물 효과	잔차
10	-7	-7.5
???	-3	-3.5
21	7	6.5
25	8	7.5
5	-5	-5.5
???	-2	-2.5

-7.5, -3.5, 6.5, 7.5, -5.5, 2.5

XGBoost 트리를 만들 때
늘 하던 대로
모든 잔차를
잎 하나에 넣을 수 있다.

결측값이 있는 새 행은 학습한 경로를 따른다

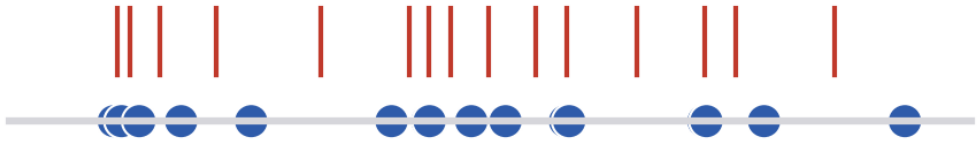
왼쪽: 결측 행은 이득이 큰 쪽으로 간다. 오른쪽: 새 행도 그쪽을 쓴다



XGBoost는 절단을 적게 시험하고 모든 코어를 쓴다

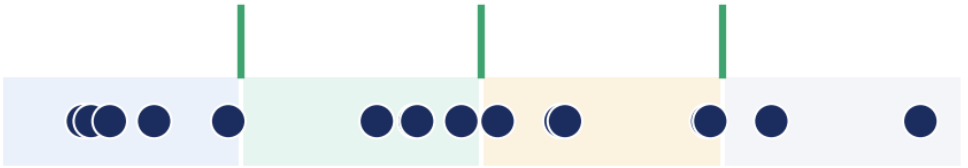
히스토그램 방식: 값을 먼저 구간으로 묶으므로 피처마다 시험할 절단이 적다

고전적 GBM: 모든 간격 시험



16행에 후보 절단 15개
(100만 행 → 절단 약 100만 개)

XGBoost: 구간 경계만 시험



행 수와 무관하게 후보 절단 3개
(기본값 약 256개 구간)

고전적 GBM: 코어 하나, 피처를 하나씩



시간 ≈ 블록 6개

XGBoost: 모든 코어를 동시에

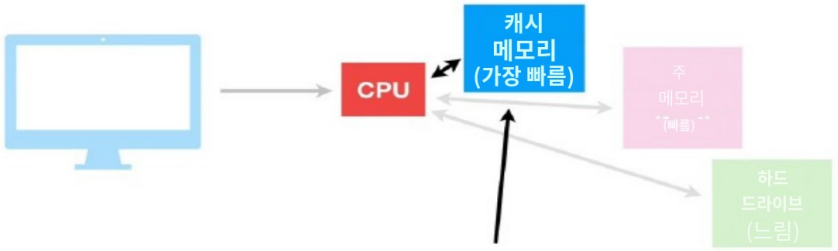


코어 1 · 코어 2 · 코어 3

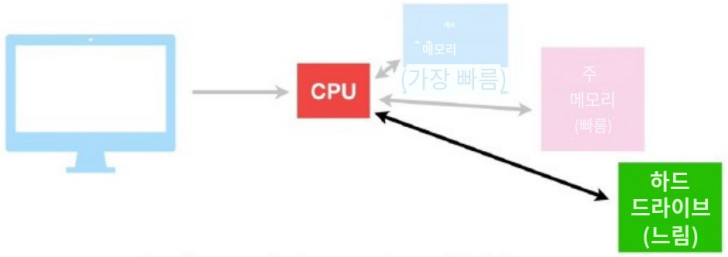
시간 ≈ 블록 2개

캐시 인식 접근으로 CPU를 쉬지 않게 한다

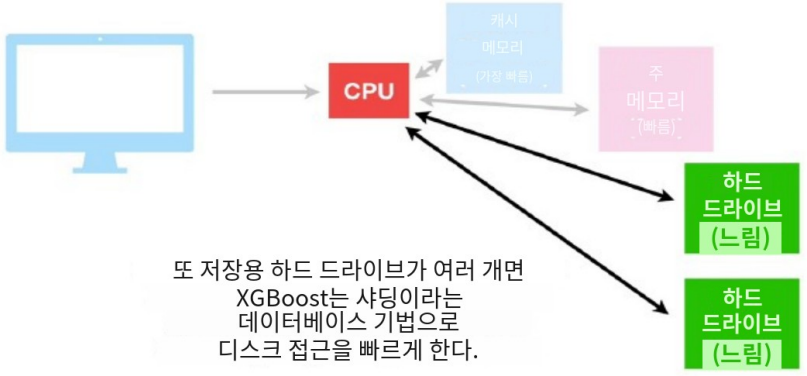
그래디언트는 빠른 캐시 메모리에 두고, 큰 데이터는 압축해 여러 하드 드라이브에 나눈다



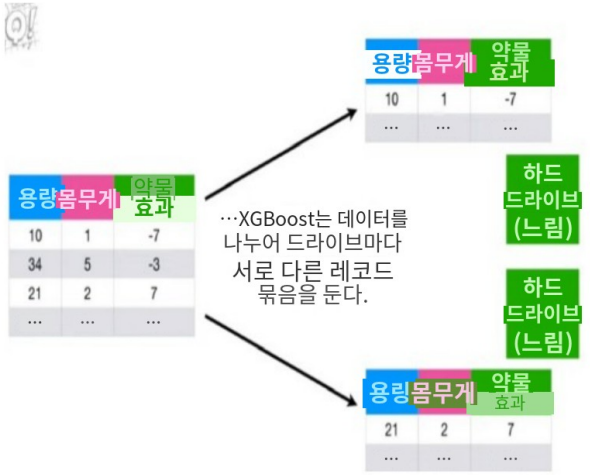
그래서 XGBoost는 그래디언트와
헤시안을 캐시에 두어
유사도 점수와 출력값을
빠르게 계산한다.



즉, 데이터 압축을 푸는 데
CPU 시간을 조금 쓰면
하드 드라이브 접근에 드는
많은 시간을 피할 수 있다.



또 저장용 하드 드라이브가 여러 개면
XGBoost는 샤딩이라는
데이터베이스 기법으로
디스크 접근을 빠르게 한다.



XGBoost도 행과 열을 무작위로 표본추출한다

랜덤 포레스트처럼 트리마다 무작위 부분집합을 보므로 학습이 빨라지고 과적합이 줄어든다

용량	질량 (kg)	좋아하는 숫자	기타 항목	약물 효과
10	63	32132	...	-7
34	72	12	...	-3
21	55	1001	...	7
...	...	...	...	...

마지막으로, XGBoost는
트리마다 데이터의 무작위 부분집합만 써서 만들게 해 속도를 높일 수도 있다.

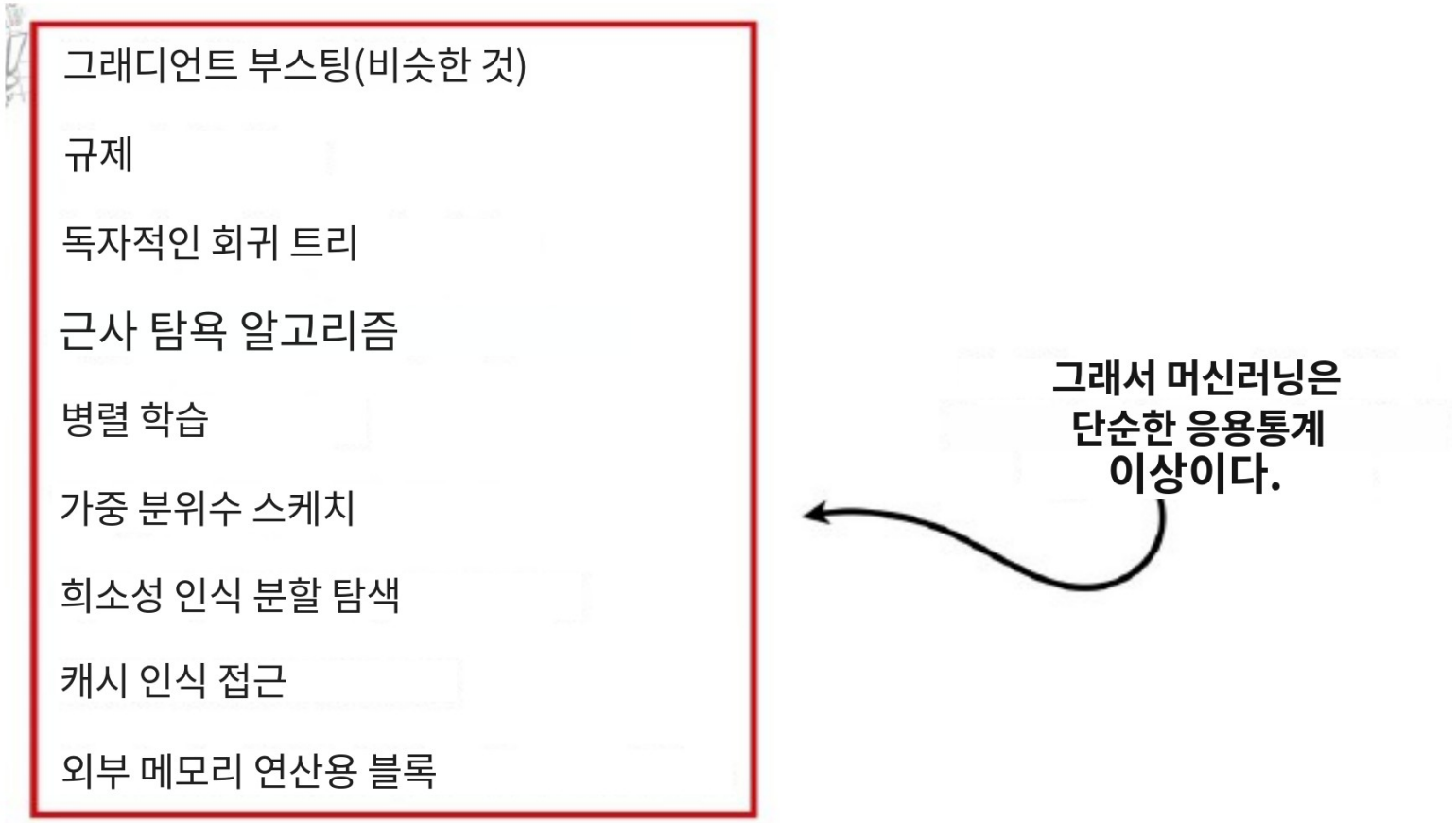


XGBoost는 부스팅에 시스템 공학을 크게 더한 것이다

라이브러리 하나에 담긴 아이디어 전체 목록

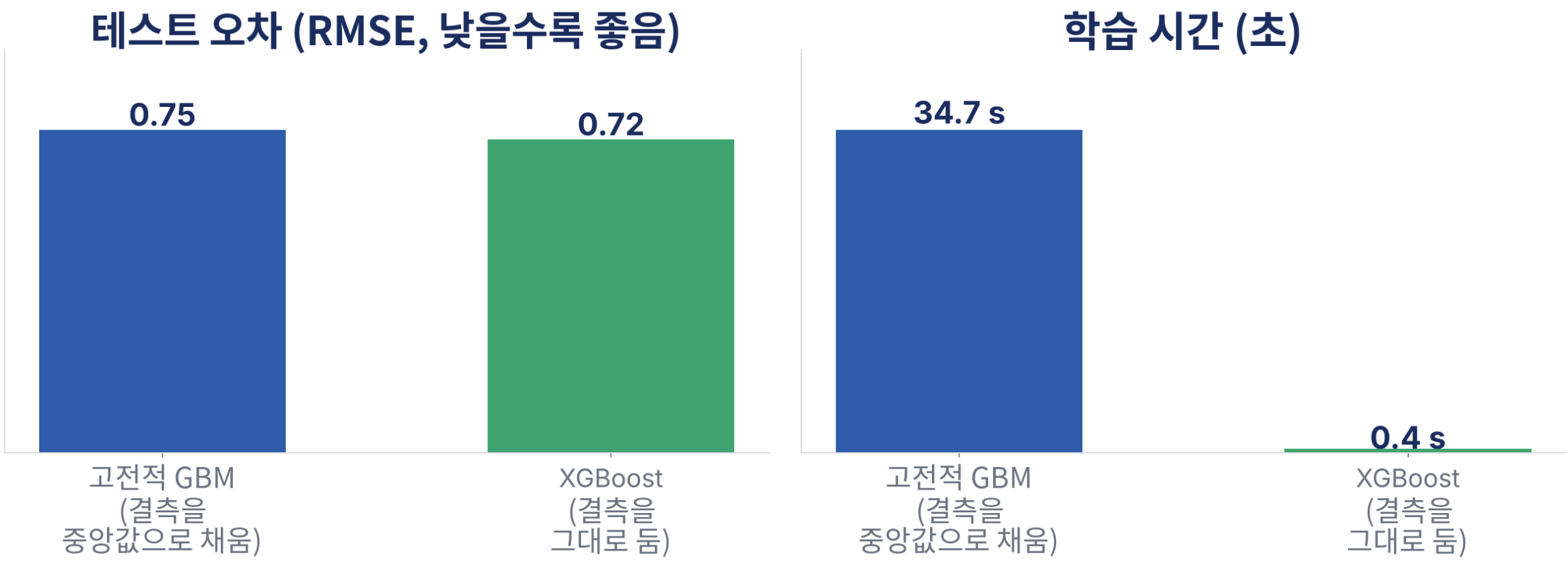
정리

- 통계: 벌점을 단 부스팅.
- 공학: 스케치, 병렬 처리, 영리한 메모리 사용.
- 머신러닝은 응용통계 이상이다.



정면 비교: 조금 더 정확하고 훨씬 빠르다

30,000행, 한 피처의 20% 결측, 각각 깊이 4 트리 400개. 시간은 이 컴퓨터 기준



에피소드

XGBoost는 물리학 대회로 유명해졌다

2014년 Kaggle 힉스 보손 머신러닝 챌린지

한 학생의 오픈소스 라이브러리가 대회 하나에서 시작해 표 형식 리더보드 대부분으로 퍼졌다

우승 비결

- 여러 설정을 시도할 만큼 빨랐다.
- 벌점이 트리가 잡음을 외우지 않게 막았다.
- 결측값을 추가 작업 없이 처리했다.

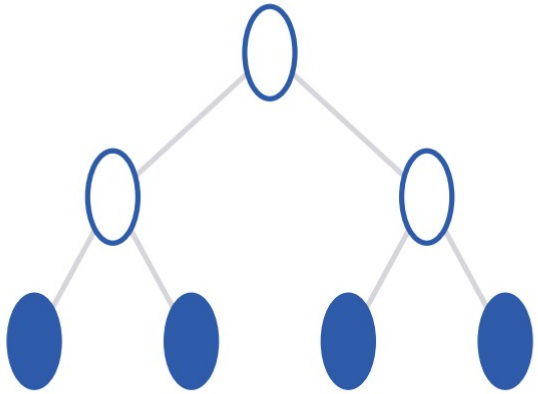
그 뒤

- 2017년 마이크로소프트가 LightGBM을, 얀덱스가 CatBoost를 공개했다.
- 세 라이브러리 모두 지금은 산업계 표준 도구다.

세 라이브러리는 서로 다른 모양으로 트리를 키운다

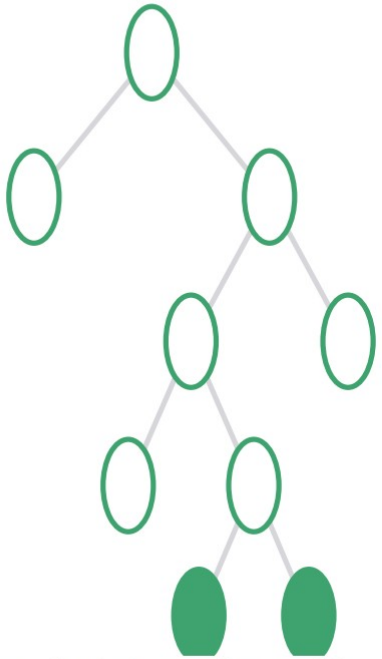
채운 원 = 가장 최근 단계에서 분할된 노드

레벨 단위 (XGBoost)



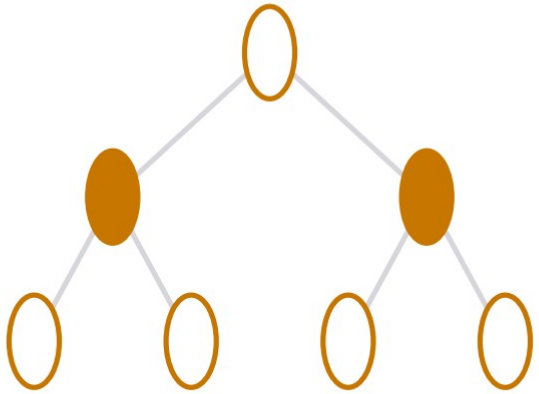
한 번에 한 레벨씩 키운다

잎 단위 (LightGBM)



이득이 가장 큰 잎을 분할

대칭 (CatBoost)



같은 질문: x3 < 0.7 ?

한 레벨의 모든 노드에 같은 규칙

LightGBM은 그래디언트 부스팅에 두 가지 기법을 더한다

GOSS는 행을 영리하게 표본추출하고, EFB는 희소 열을 묶는다

LightGBM

= GBDT (그래디언트 부스팅 결정 트리)

\+ GOSS (그래디언트 기반 단측 표본추출)

\+ EFB (배타적 피처 묶음)

분할은 데이터 히스토그램으로 점수를 매긴다

타이타닉 예시: 나이를 예측. 생존 여부와 성별로 나눈 뒤 남는 가중 분산을 비교한다

타이타닉 데이터로 승객의 나이를 예측한다.

성별	생존	나이
여	1	29
남	1	1
여	0	2
남	0	30
여	0	25
남	1	48
여	1	63
남	0	39
여	1	53
남	0	71

분산: 498.29

분산 계산

생존	분산	인원
0	502.64	5
1	479.36	5

성별	분산	인원
남	524.56	5
여	466.24	5

가중평균
분산

491.0 — 7.29 감소

495.4 — 2.11 감소

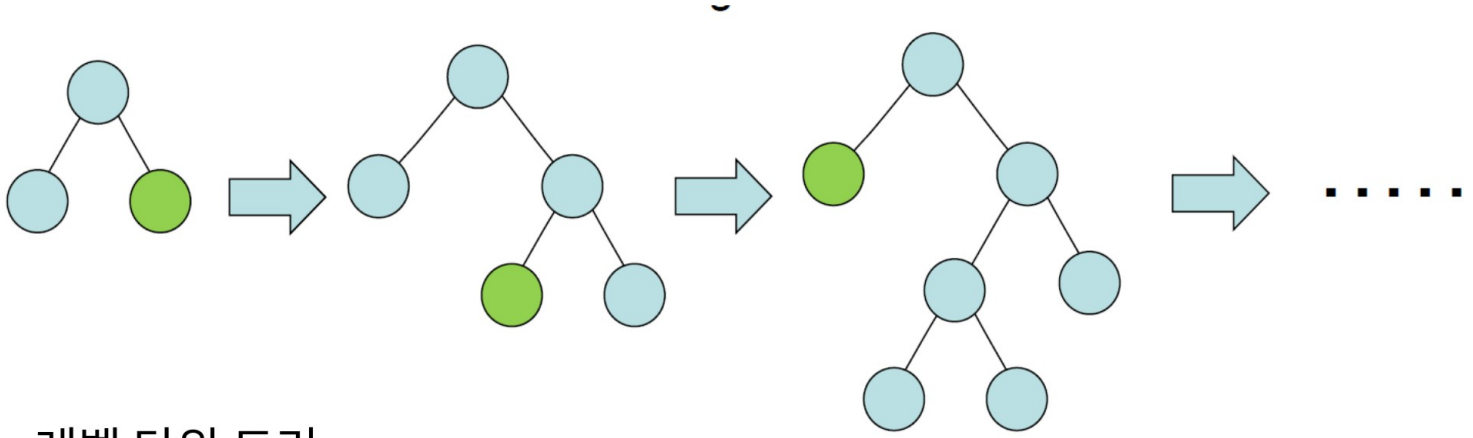
LightGBM은 가장 유용한 잎부터 키운다

위: 잎 단위(leaf-wise) 성장. 아래: XGBoost가 쓰는 레벨 단위(level-wise) 성장

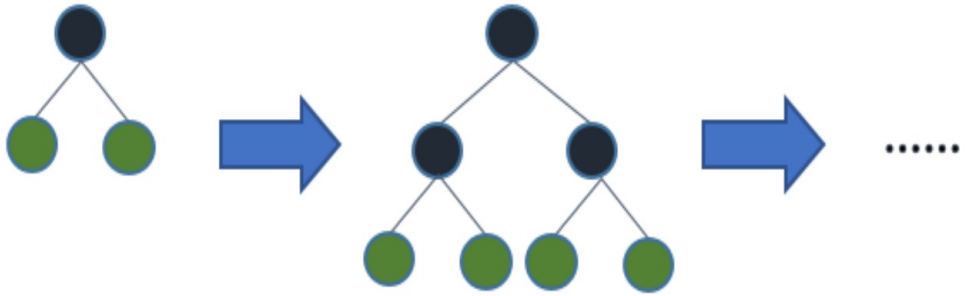
왜 중요한가

- 잎 단위 트리는 더 적은 분할로 낮은 오차에 이른다.
- 깊고 한쪽으로 치우치게 자랄 수 있다.
- 데이터가 작으면 num_leaves나 깊이를 제한한다.

- 잎 단위 트리 (깊이 우선 트리)

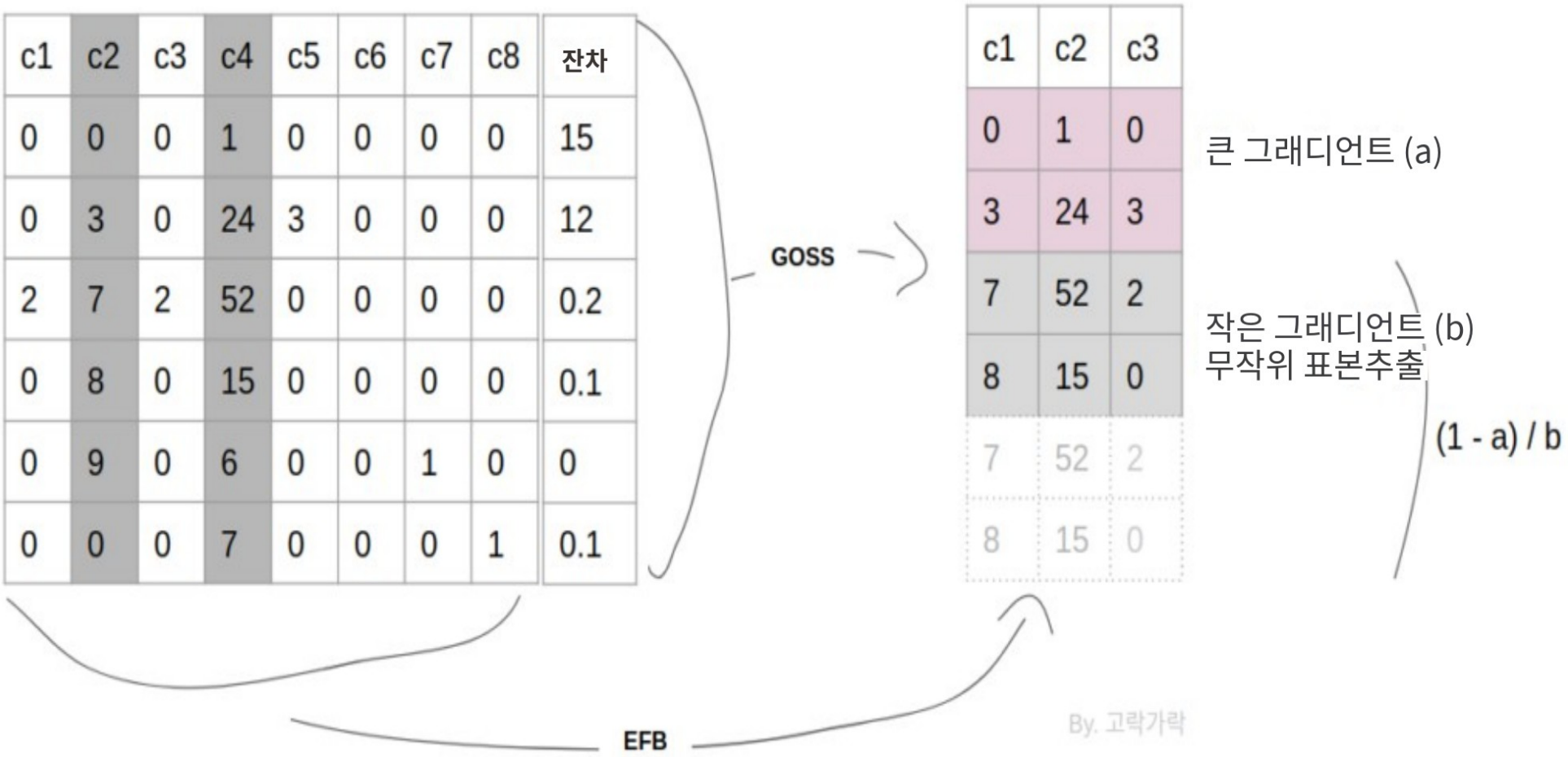


- 레벨 단위 트리



GOSS와 EFB는 두 방향에서 데이터를 줄인다

GOSS는 쉬운 행을 덜어 내고(행 감소), EFB는 희소 열을 합친다(열 감소)



GOSS는 어려운 행은 남기고 쉬운 행은 표본추출한다

그래디언트 크기로 행을 정렬해 상위 a × 100%는 남기고, 나머지에서 b × 100%를 뽑아 가중치를 키운다



https://cdm98.tistory.com/m/31

LightGBM은 오차가 아직 큰 행으로 학습한다

GOSS = 그래디언트 기반 단측 표본추출(Gradient-based One-Side Sampling): 어려운 행은 남기고 쉬운 행은 표본추출



큰 원 =
큰 오차

모형이 아직
몇 행에서 틀린다

큰 오차는 모두 남기고,
작은 것 9개 중 3개 뽑기

뽑은 작은 행은 ×3으로 세어
9개를 대신한다

LightGBM은 동시에 0이 아닌 적이 없는 열을 합친다

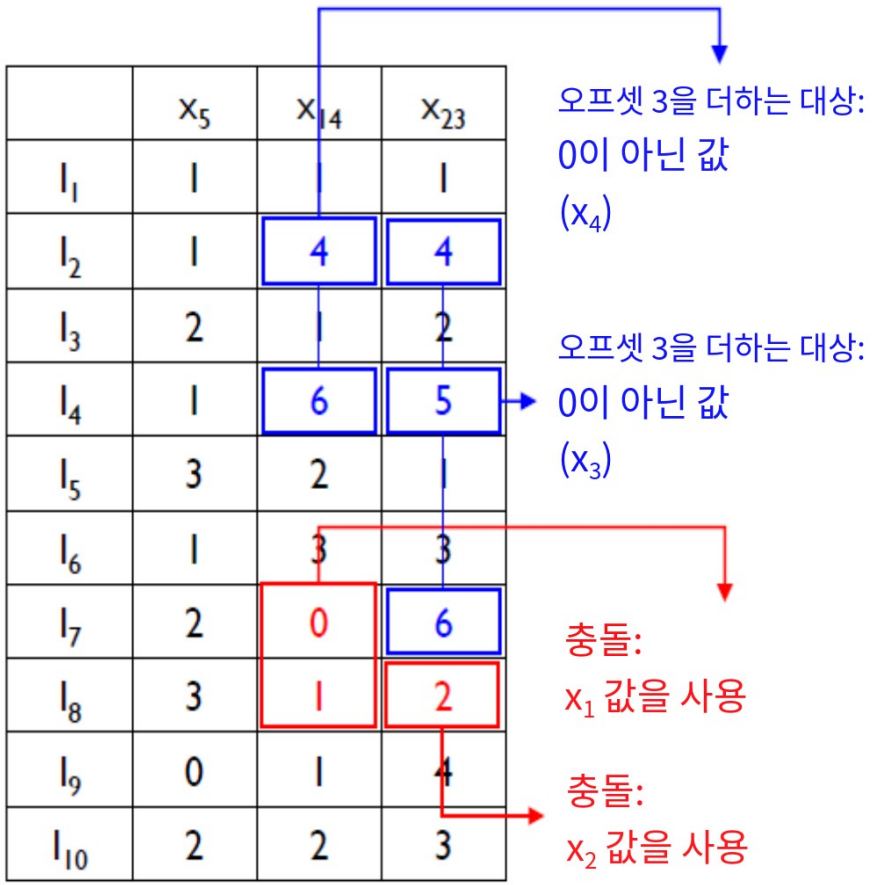
EFB = 배타적 피처 묶음(Exclusive Feature Bundling): 희소한 원-핫 열을 한 열로 만든다



EFB는 오프셋을 더해 묶인 값이 겹치지 않게 한다

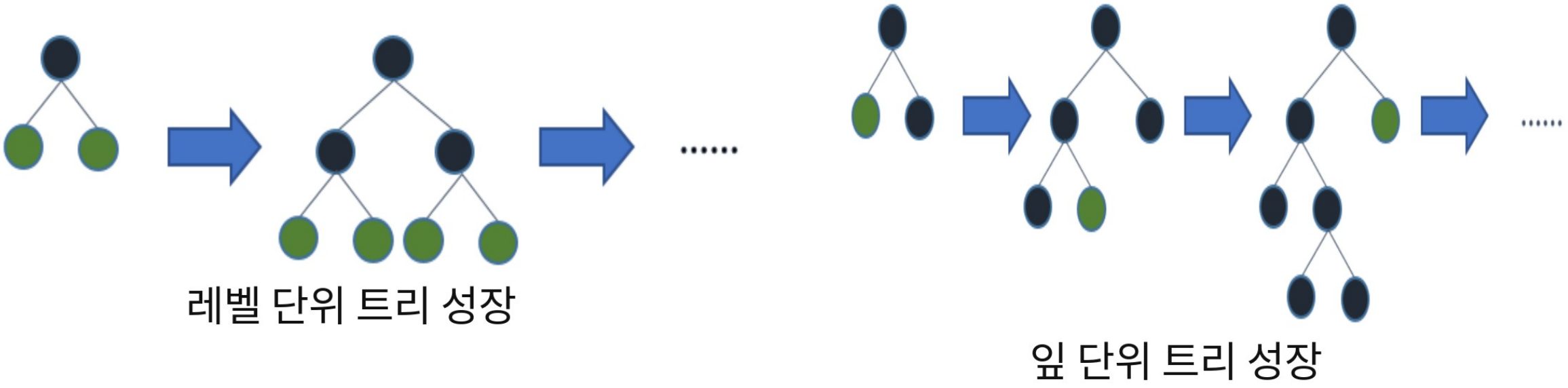
원 논문 예시: x4의 값을 3만큼 옮긴 뒤 x1과 합친다. 드문 충돌에서는 한 값만 남긴다

	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0



CatBoost는 레벨 단위 대칭 트리를 만든다

왼쪽: 레벨 단위 성장(XGBoost, CatBoost). 오른쪽: 잎 단위 성장(LightGBM)



단순 타깃 인코딩은 정답을 누출한다

날씨 범주로 그날 최고 기온을 예측한다

문제

- "흐림"을 흐린 날의 평균 기온 18.5로 바꾼다.
- 그 평균에는 각 행 자신의 정답이 이미 들어 있다.
- 모형이 예측 시점에는 없을 값으로 학습한다.

요일	피처 1	클래스 라벨 (그날 최고 기온)
일요일	맑음	35
월요일	맑음	32
화요일	흐림	15
수요일	흐림	14
목요일	대체로 흐림	10
금요일	흐림	20
토요일	흐림	25

CatBoost는 앞선 행만으로 범주를 인코딩한다

순서형 타깃 통계(ordered target statistics): 행이 자기 타깃을 보지 않으므로 타깃 누출을 막는다

행	색	타깃	앞선 행 중 같은 색	인코딩 값
1	빨강	1	아직 없음	0.50
2	파랑	0	아직 없음	0.50
3	빨강	0	타깃 1	0.75
4	빨강	1	타깃 1, 0	0.50
5	파랑	1	타깃 0	0.25
6	빨강	1	타깃 1, 0, 1	0.62

CatBoost는 함께 움직이는 범주를 결합한다

순서형 피처 결합: 여기서는 국가와 머리색이 같은 정보를 담는다

하는 일

- 국가만으로 머리색이 정해진다.
- CatBoost는 학습 중 범주 결합을 시험한다.
- 사용자가 결합을 직접 만들 필요가 없다.

국가	머리색	클래스 라벨
인도	검정	1
인도	검정	1
인도	검정	1
인도	검정	1
러시아	흰색	0
러시아	흰색	0
러시아	흰색	0
러시아	흰색	0

라이브러리마다 기본 강점이 다르다

일반적인 기본 설정 기준. 셋 다 조정하면 비슷하게 동작할 수 있다

	XGBoost	LightGBM	CatBoost
트리 성장	레벨 단위	잎 단위	대칭
결측값	학습한 방향	학습한 방향	내장 처리
범주형 데이터	인코딩 필요*	기본 지원	순서형 통계
강점	두루 안정적	큰 데이터에서 속도	조정이 거의 필요 없음

* 최신 XGBoost는 실험적 범주형 기본 지원을 추가했다

최신 라이브러리는 고전적 부스팅만큼 정확하고 훨씬 빠르다

프리드먼(Friedman) 시뮬레이션 데이터, 학습 30,000행, 트리 500개. 시간은 컴퓨터에 따라 다르다

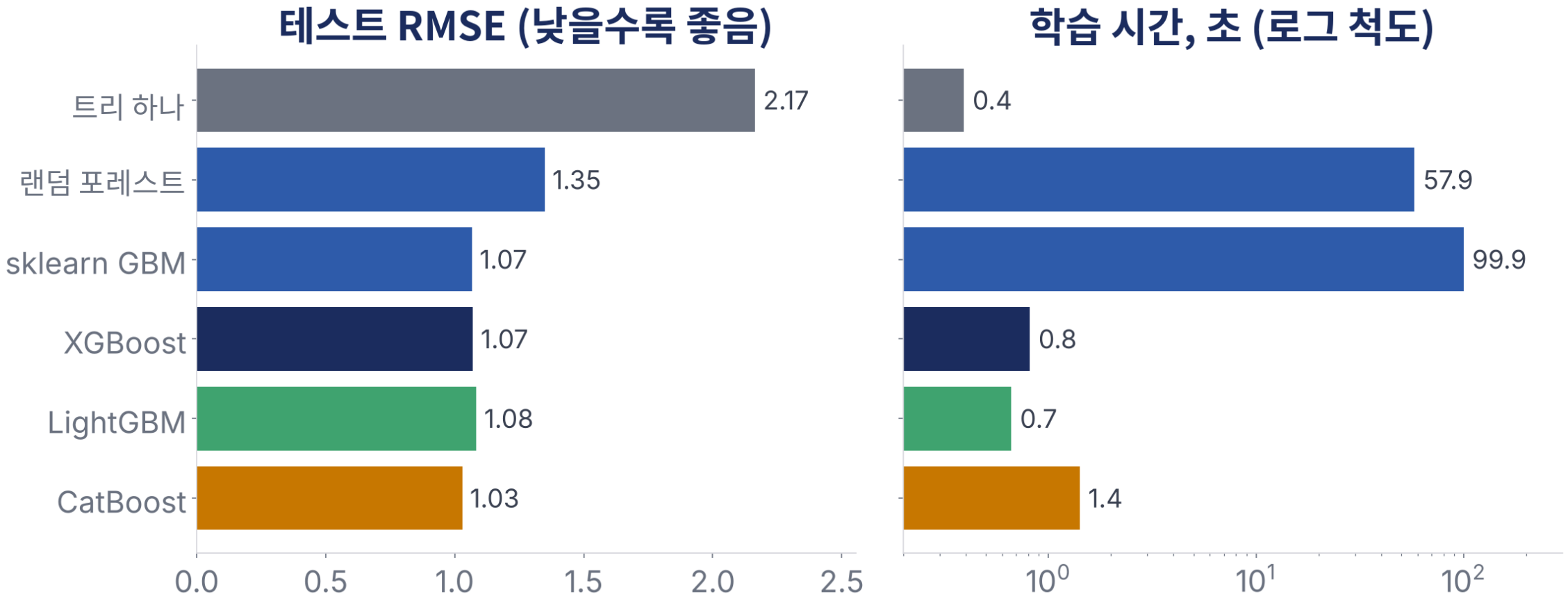


표 형식 데이터에서는 여전히 트리 앙상블이 앞선다

그린슈타인·오얄롱·바로코(Grinsztajn, Oyallon and Varoquaux, NeurIPS 2022)가 데이터셋 45개로 모형을 비교했다

중간 크기 표 데이터에서는 조정한 트리 앙상블이 대부분의 데이터셋에서 심층 신경망보다 우수했다

표 데이터에서 트리가 이기는 이유

- 실제 목표값은 매끄럽지 않고 급격히 변한다.
- 정보 없는 열이 많은데 트리는 이를 무시한다.
- 스케일 조정이나 세심한 전처리가 필요 없다.

딥러닝이 이기는 곳

- 원시 입력에 구조가 있는 이미지, 음성, 텍스트.
- 매우 큰 데이터셋, 그리고 텍스트 필드가 섞인 입력.

05

5부 · 트리 모형 해석

피처 중요도와 SHAP

모형은 어떤 입력을 쓰며, 이 예측은 왜 나왔나?

피처 중요도는 어떤 입력이 중요한지 보여 준다

흔히 쓰는 도구 세 가지, 각각 조금씩 다른 질문에 답한다

분할 기반

트리에 내장

이 피처로 분할해 노드가 얼마나 순수해졌나?

빠르지만 편향

순열

섞은 뒤 다시 평가

이 피처를 뒤섞으면 모형이 얼마나 나빠지나?

정직하지만 전역만

SHAP

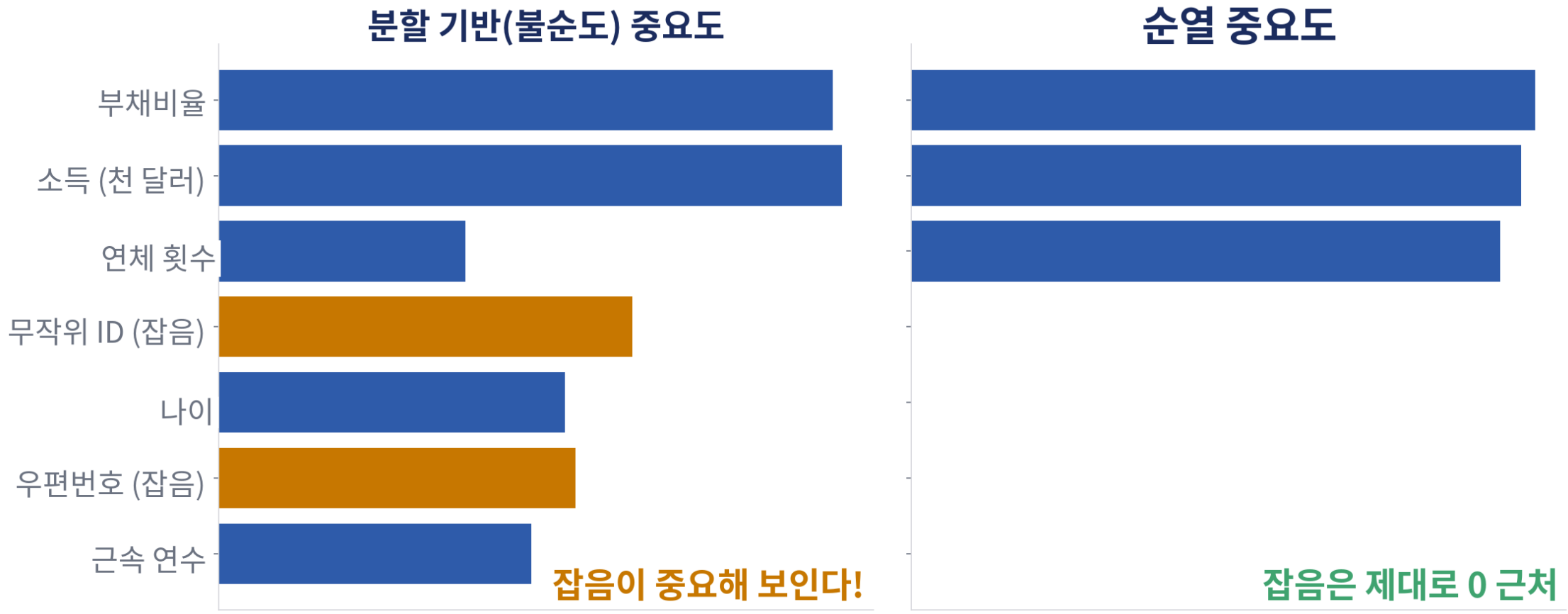
행별 공정한 기여

이 피처가 이 예측 하나를 얼마나 올리거나 내렸나?

전역과 국소 모두

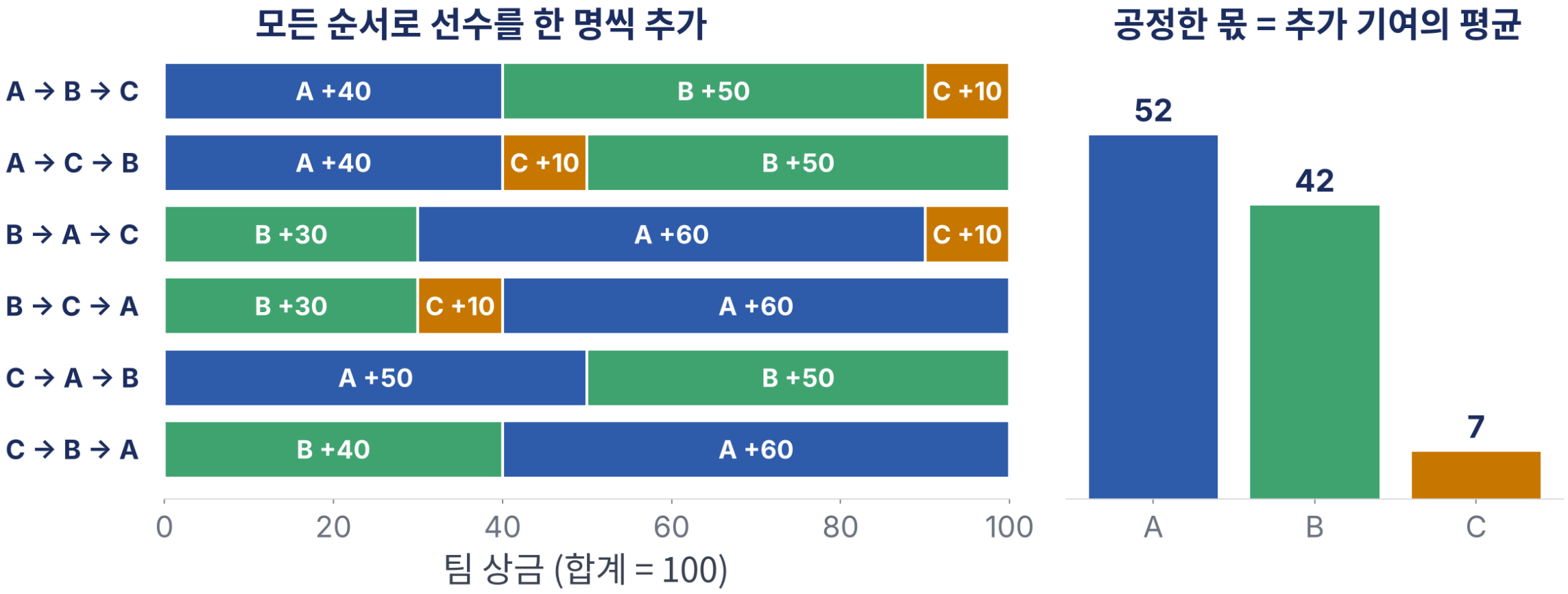
분할 기반 중요도는 잡음에 속을 수 있다

순수 잡음 열 2개를 넣은 대출 시뮬레이션 데이터. 중요도는 따로 떼어 둔 행에서 측정



샤플리 값은 팀 상금을 공정하게 나눈다

게임이론(로이드 샤플리, 2012 노벨상): 참가 순서 전부에 대해 각 선수의 추가 기여를 평균한다



SHAP은 피처를 팀의 선수로 본다

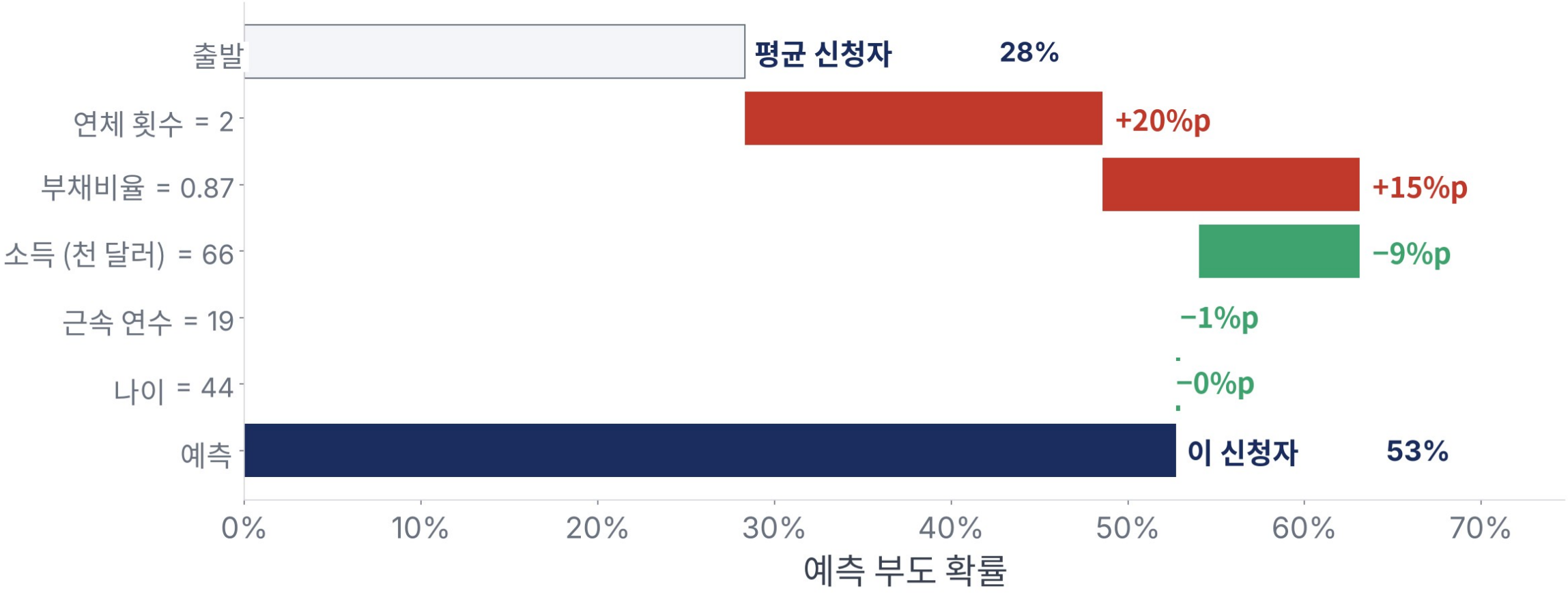
SHAP = SHapley Additive exPlanations(Lundberg and Lee, 2017)

팀 게임에서	트리 모형에서
선수	한 신청자의 피처
팀 상금	예측값 – 평균 예측값
각 선수의 공정한 몫	각 피처의 SHAP 값
몫을 더하면 상금	SHAP 값을 더하면 예측값

트리 모형에서는 TreeSHAP이 이 몫을 정확하고 빠르게 계산한다

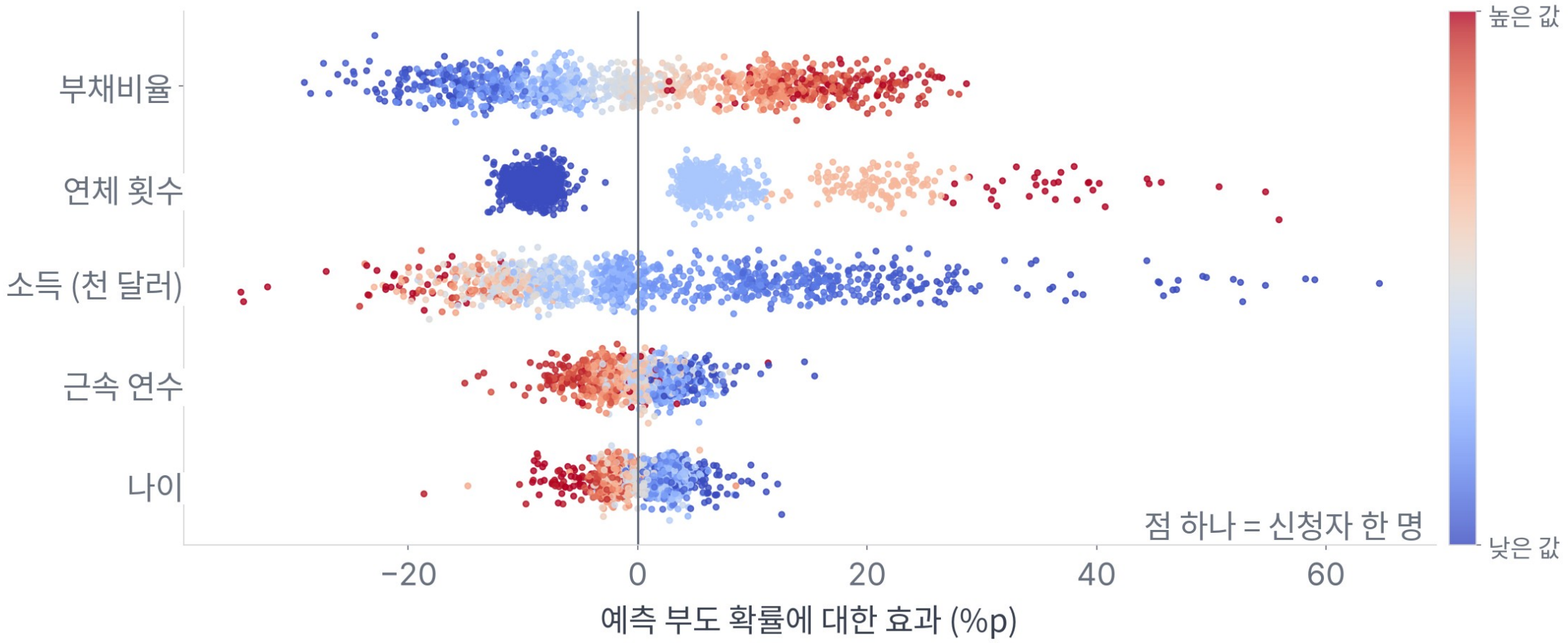
SHAP은 예측 하나를 피처별로 설명한다

대출 신청자 한 명: 평균 부도 확률에서 출발해 피처별 기여를 더한다



요약 그림은 크기와 방향을 함께 보여 준다

점 하나가 신청자 한 명. 빨강 = 피처 값 높음, 파랑 = 낮음, 오른쪽 = 위험 증가

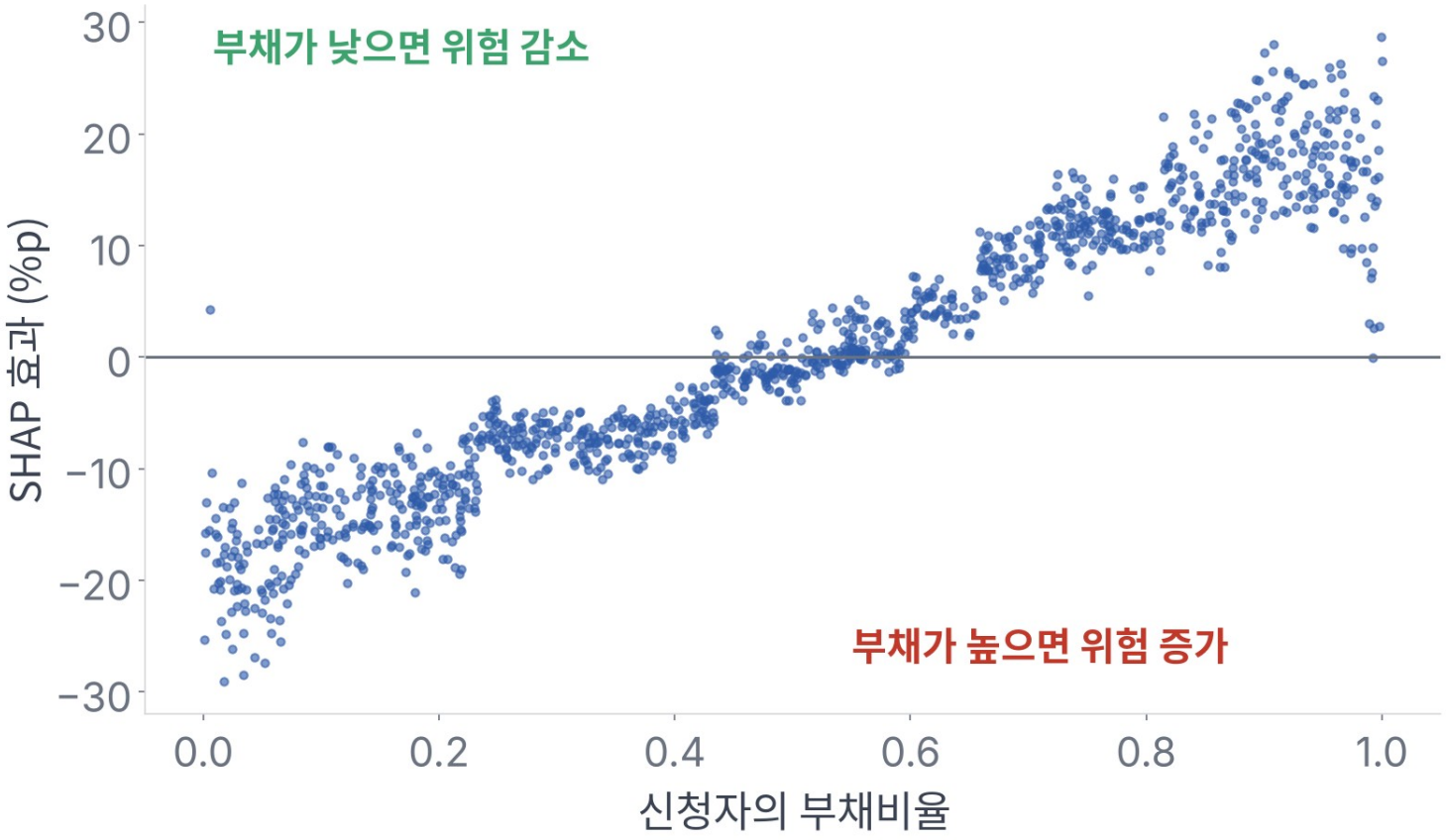


의존도 그림은 한 피처가 위험을 어떻게 움직이는지 보여 준다

부채비율과 그 SHAP 효과, 점 하나가 신청자 한 명

관찰

- 약 0.5 아래에서는 부채가 위험을 낮춘다.
- 그 위에서는 위험이 꾸준히 오른다.
- 모형이 이 형태를 스스로 학습했다.



대출 기관은 거절 이유를 설명해야 한다

미국 대출 기관은 거절의 주된 이유를 적은 거절 사유 통지서(adverse action notice)를 보낸다

SHAP 값으로 모형의 결정을 신청자별 이유 순위 목록으로 바꿀 수 있다

활용 방식

- 거절 쪽으로 미는 SHAP 기여 순으로 피처를 정렬한다.
- 상위 몇 개를 사유 코드로 보고한다.
- 부당한 피처가 결정을 좌우하지 않는지 점검한다.

기억할 한계

- SHAP은 현실이 아니라 모형을 설명한다.
- 상관된 피처는 기여를 이상하게 나눠 가질 수 있다.
- 인과관계의 증거가 아니다.

표 형식 문제 대부분에 통하는 간단한 절차

단순하게 시작하고, 검증 오차가 개선될 때만 복잡도를 더한다

1 기준선

랜덤 포레스트

조정할 설정이 적다. OOB 오차를 확인한다.

트리 300–500개

2 부스팅

최신 GBDT 아무것이나

작은 학습률과 조기 종료.

학습률 0.05–0.1

3 조정

복잡도부터

깊이 또는 잎 수, 최소 자식 크기, 행·열 표본추출.

교차검증

4 해석

배포 전에

순열 중요도 또는 SHAP 값.

누출 점검

토론 질문

정답은 하나가 아니다. 선택의 근거를 제시하라

Q1	은행은 거절마다 이유를 설명해야 한다. 작은 트리 하나와 SHAP을 붙인 부스팅 모형 중 무엇을 쓰겠는가?	**해석 가능성**
Q2	포레스트의 학습 오차가 1%, 테스트 오차가 20%다. 어떤 설정부터 바꾸겠는가?	**과적합**
Q3	부스팅은 트리를 더할수록 과적합하는데 랜덤 포레스트는 왜 그렇지 않은가?	**편향 대 분산**
Q4	데이터에 수준이 5,000개인 범주(매장 ID)가 있다. 어떤 라이브러리를 먼저 써 보겠으며, 그 이유는?	**범주형 데이터**

요약

트리는 단순하지만 그 앙상블은 강력하다

기억할 것

1

트리는 데이터를 상자로 나눈다

읽기 쉽지만 분산이 크다. 깊이를 제한하거나 여러 트리를 평균한다.

2

포레스트는 평균하고 부스팅은 고친다

랜덤 포레스트는 분산을 줄이고, 부스팅은 작은 트리와 작은 학습률로 편향을 줄인다.

3

SHAP이 블랙박스를 연다

섀플리 값은 예측 하나에 대한 각 피처의 공정한 기여를 주고, 모으면 전역 그림이 된다.

다음: XGBoost와 SHAP 실습